



PROGRAMACIÓN ORIENTADA A OBJETOS

FACULTA DE INGENIERÍA EN INFORMÁTICA
ESCUELA POLITÉCNICA NACIONAL



CLASS GROUP



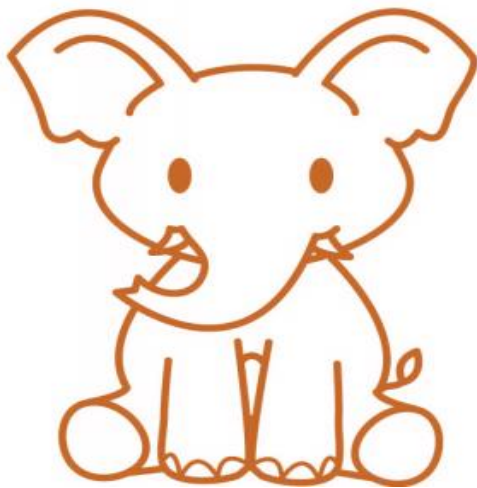
CLASS ROOM

Patricio Michael Paccha Angamarca

Master en ingeniería de software

Diplomado Superior de Cuarto Nivel en Gerencia Estratégica de Mercadeo

Ingeniero en Sistemas Informáticos Y Computación



TROMPUD@



chuki



< toxic../>



HERRAMIENTAS DE PROGRAMACIÓN

1.1 La máquina virtual, entorno típico de programación

1.2 Tipos de datos. Reglas del lenguaje.

1.3 Diseño Orientado a Objetos: Descomposición en objetos, su estado y su comportamiento, jerarquía de clases

1.4 La clase: atributo, métodos y constructores; Control de acceso a miembros de la clase.

PROGRAMACIÓN

2.1 Efectos no considerados en las llamadas a métodos

2.2 Variables inmutables, prevención de cambios a datos por otro código

2.4 Llamadas a métodos, argumentos, iteradores

2.5 Manejo de GUI: objetos, eventos, framework

2.6 Try – catch, Stack trace, Debug

2.8 Arreglos unidimensionales, multidimensionales, Cadenas, Strings, String tokenizer

IDE en la nube

IDE locales



Visual Studio Code



git



GitHub

- Entorno de desarrollo estandarizado
- Independencia de plataformas / SO
- Mejor rendimiento
- Personalización



**SQLite**
Explore and query SQLite d...
alexcvzz

**SQLite Viewer**
SQLite Viewer for VSCode
Florian Klampfer

**Live Share**
Real-time collaborative devel...
Microsoft

**Excalidraw**
Draw schemas in VS Code usi...
pomdtr

Install New JDK

Adoptium's TemurinOthers

Version

8 (LTS)

11 (LTS)

17 (LTS)

JVM

hotspot

Download
windows-x64
jdk-17.0.5+8 | 180 MB

**Extension Pack for Java** v0.25.2022110400 Pre-Release Preview
Microsoft | 15,958,761 | ★★★★★ (53)
Popular extensions for Java development that provides Java IntelliSense, debuggi...
Disable Uninstall Switch to Release Version Windows: CTRL+B
This extension is enabled globally.

**Project Manager for Java** v0.21.1 Preview
Microsoft | 15,882,865 | ★★★★★ (5)
Manage Java projects in Visual Studio Code
Disable Uninstall Switch to Pre-Release Version
This extension is enabled globally.

**Draw.io Integration**
This unofficial extension integrates Draw.i...
Henning Dieterichs

**Draw.io Integration - Insiders Build**
This is the unstable Insiders Build
Henning Dieterichs

**Drawio Preview**
Preview drawio diagram file in vscode.
purocean

**Makefile Tools**
Provide makefile support in VS Code: C/C...
Microsoft

**Markdown PDF**
Convert Markdown to PDF
yzane

**markdownlint**
Markdown linting and style checking for ...
David Anson



Shortcuts

1. Command Palette

Windows: CTRL+SHIFT+P

2. Quick Open

Windows: CTRL+P

3. Toggle Sidebar

Windows: CTRL+B

4. Multi-Select Cursor

Windows: CTRL+D



Visual Studio Code

7. Go back / move forward

Windows: ALT+ —> or + —>

5. Copy Line

Windows: SHIFT+ALT+UP or SHIFT+ALT+DOWN

8. Show All Symbols

Windows: CTRL+T

6. Comment Code Block

Windows: SHIFT+ALT+A (Multi-line comment), CTRL+K+C (Single-line comment)

9. Trigger suggestion and Trigger parameter hints

Windows: Ctrl+Space, Ctrl+Shift+Space



Markdown



Es un lenguaje de marcado con el que puedes agregar formato a documentos de texto plano. Fue creado por [John Gruber](#) en el año 2004, siendo a día de hoy uno de los lenguajes de marcado más famosos



Extensión

La extensión **.md**, **.markdown** o incluso **.mdx** en caso de que quieras agregar funcionalidades adicionales.

Titulo # ## ### #### #####

****palabras en negrita****

palabras en cursiva

******palabras en negrita y cursiva******

==Texto resaltado==

~~Texto tachado~~





! [[EPN](#)] (img/uno2.png)

> comando para git

- \$ git version
- \$ pwd
- \$ git init

```
` `` `c
int main()
{
    printf("hola pat_mic...!");
    return 0;
}
` `` `
```

Link

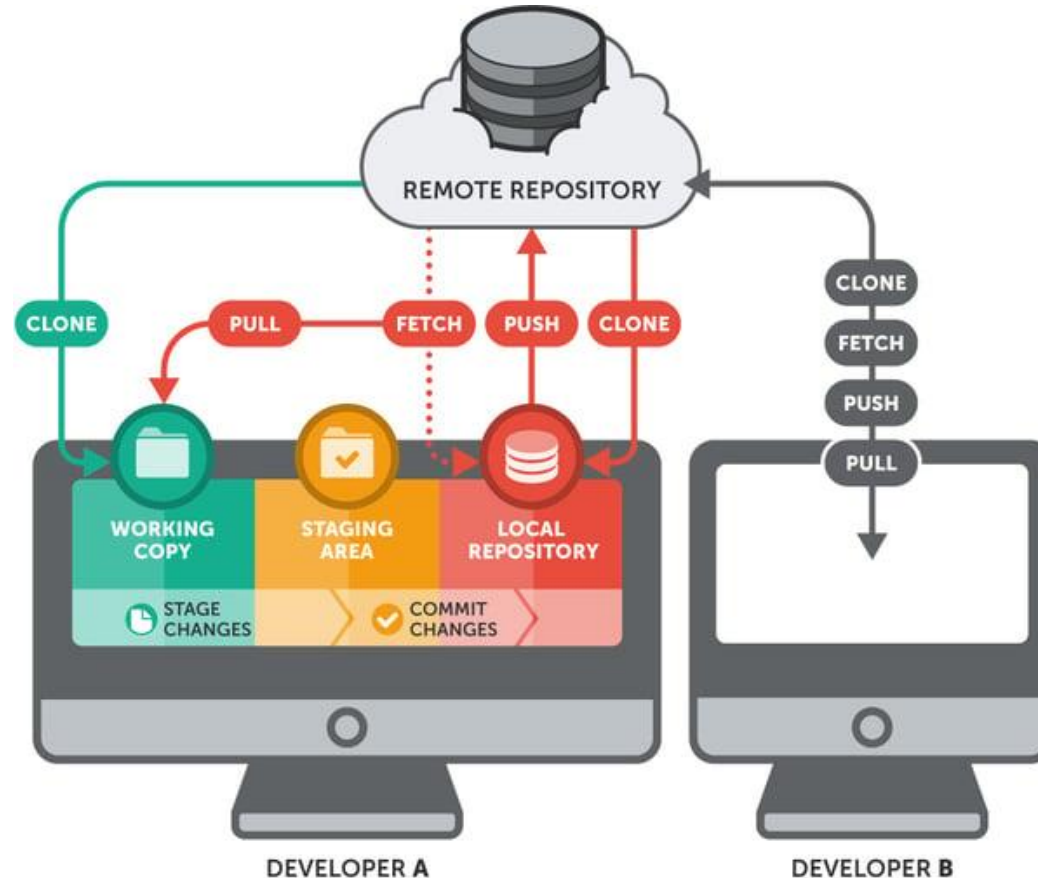
[[buscador google](#)] (<http://google.com>)







Git fue creado en 2005 por el mismo Linus Torvalds ([el creador de Linux](#)) **como herramienta para facilitar el desarrollo colaborativo** de software. Desde entonces han surgido incluso populares plataformas, [como GitHub o GitLab](#), que permiten su uso online.





```
$ git --version
```

Tu Identidad

```
$ git config --global user.name "pat_mic"
```

```
$ git config --global user.email pat_mic@hotmail.com
```

Verificando

```
$ git config user.name
```

```
$ git config user.email
```

inicio del control de versiones

```
$ git init
```

```
$ git status
```

```
$ git add .
```

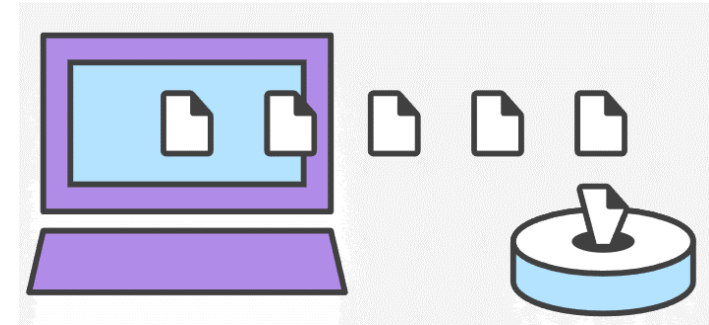
```
$ git add NombreCarpeta/NombreArchivo.ext
```

```
$ git commit -m 'mensaje : initial project version'
```

inicio del control de versiones - clonando

```
$ git clone https://github.com/xyyyy/abc
```

```
$ git clone https://github.com/xyyyy/abc miPropioNombre
```



Touch (crear archivos)

Quitar archivos del control

```
$ echo "nombreArchivo.ext" >> .gitignore
```

```
$ echo "*.txt" >> .gitignore
```

Ver archivos ignorados

```
$ cat .gitignore
```

Forzar agregar archivo excluido

```
$ git add -f NombreArchivo.log
```



create & clone

create new repository	<code>git init</code>
clone local repository	<code>git clone /path/to/repository</code>
clone remote repository	<code>git clone username@host:/path/to/repository</code>

add & remove

add changes to INDEX	<code>git add <filename></code>
add all changes to INDEX	<code>git add *</code>
remove/delete	<code>git rm <filename></code>

commit & synchronize

commit changes	<code>git commit -m "Commit message"</code>
push changes to remote repository	<code>git push origin master</code>
connect local repository to remote repository	<code>git remote add origin <server></code>
update local repository with remote changes	<code>git pull</code>



branches

create new branch	<code>git checkout -b <branch></code> <i>e.g. git checkout -b feature_x</i>
switch to master branch	<code>git checkout master</code>
delete branch	<code>git branch -d <branch></code>
push branch to remote repository	<code>git push origin <branch></code>

merge

merge changes from another branch	<code>git merge <branch></code>
view changes between two branches	<code>git diff <source_branch> <target_branch></code> <i>e.g. git diff feature_x feature_y</i>

tagging

create tag	<code>git tag <tag> <commit ID></code> <i>e.g. git tag 1.0.0 1b2e1d63ff</i>
get commit IDs	<code>git log</code>

restore

replace working copy with latest from HEAD	<code>git checkout -- <filename></code>
--	---

Tip

Want a simple but powerful
git-client for your mac?

Try Tower: www.git-tower.com/





Git es el sistema de control de versiones más utilizado del mundo.

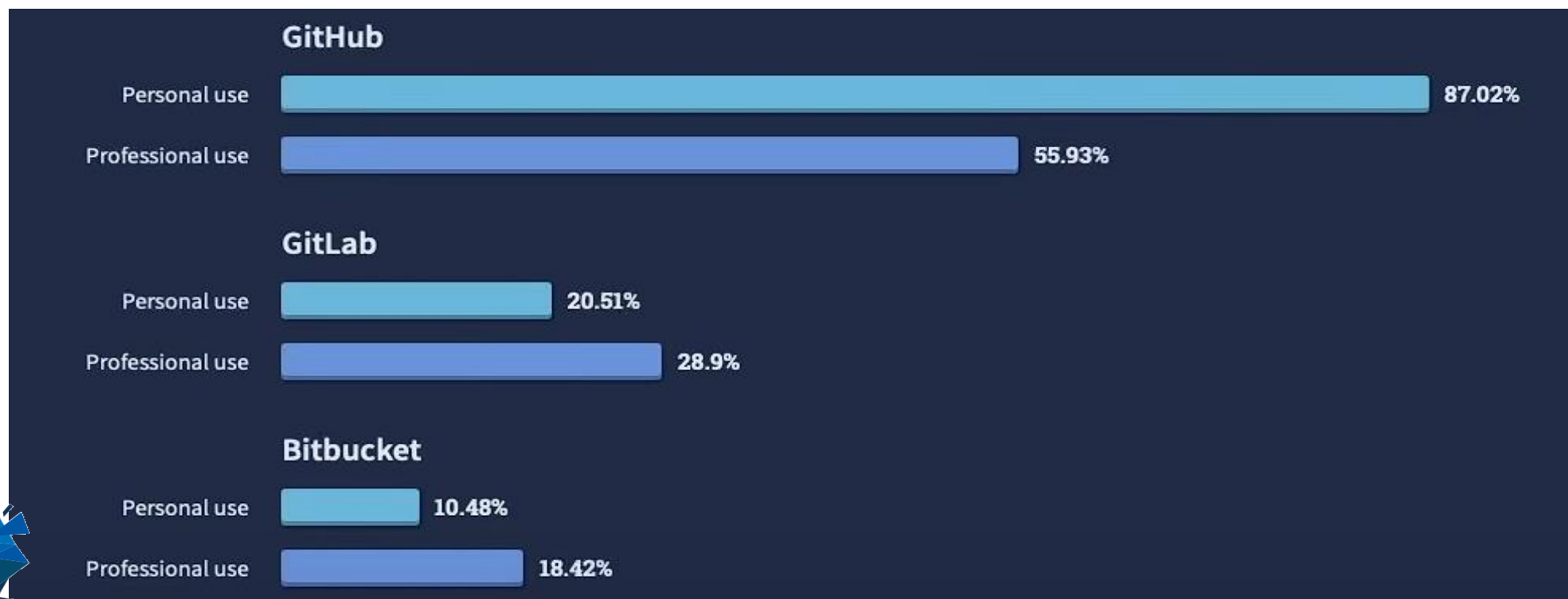
Git es un proyecto de código abierto maduro y con un mantenimiento activo que desarrolló originalmente Linus Torvalds, el famoso creador del kernel del sistema operativo Linux, en 2005.

Un asombroso número de proyectos de software dependen de Git para el **control de versiones**, incluidos proyectos comerciales y de código abierto y este sistema funciona a la perfección en una amplia variedad de sistemas operativos e IDE (entornos de desarrollo integrados).





GitHub es una plataforma más popular de control de versiones para alojar el código de las aplicaciones de cualquier desarrollador, y que fue comprada por Microsoft [en junio del 2018](#). La plataforma está creada para que **los desarrolladores suban el código de sus aplicaciones y herramientas**, y que como usuario no sólo puedas descargar la aplicación, sino también entrar a su perfil para leer sobre ella o colaborar con su desarrollo.





94M

developers are
on GitHub

90%

of companies use
open source*

90%+

of Fortune 100
companies use
GitHub

413M

open source
contributions in
2022

State of Open: The UK in 2021 Phase Three The Values of Open." October 2021. Open UK. Accessed August 5, 2022.





Workshop – GIT

<https://www.atlassian.com/es/git/tutorials>

<https://git-scm.com/book/es/v2>

pat_mic

```
❯ 🦴 git version  
git version 2.40.1.windows.1
```

pat_mic

```
❯ 🦴 git help  
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=  
        [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]  
        [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--n  
        [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>] [  
        [--config-env=<name>=<envvar>] <command> [<args>]
```

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)

clone	Clone a repository into a new directory
init	Create an empty Git repository or reinitialize



2nd Edition (2014)





Comandos básicos

```
pat_mic
( 🐼 ) ls -a
./ ../ .git/
pat_mic
( 🐼 ) ls -al
total 8
drwxr-xr-x 1 pat_m 197615 0 May  2 16:57 ./
drwxr-xr-x 1 pat_m 197615 0 May  2 16:59 ../
drwxr-xr-x 1 pat_m 197615 0 May  2 16:57 .git/
```

```
pat_mic
( 🐼 ) git config --global user.name
pat_mic
pat_mic
( 🐼 ) git config --global user.email
pat_mic@hotmail.com
pat_mic
( 🐼 ) git config --global user.name "TU_NOMBRE"
```

18 Most-used Linux Commands



ls

list files and directories



cd

change current directory



mkdir

create new directory

rm

remove files or directories

mv

move or rename files
or change file or directory

chmod

change file or
directories permission

cp

copy files or directories

find

search for files or directories

grep

search for a pattern in files

vi

edit files using text editor

cat

display the content of files

tar

manipulate tarball archive
files

ps

display process information

kill

terminate process by
sending a signal

top

display process and
resource usage

ifconfig

configure network
interfaces

ping

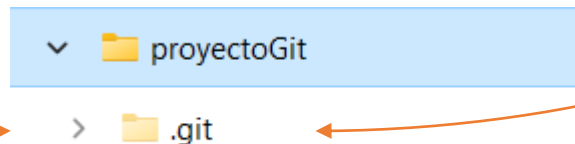
test network connectivity
between hosts

du

estimate file space usage



Crear un directorio
Abres el directorio en vscode



```
pat_mic
{ skull } pwd
/c/pat_mic/epn_code/proyectoGit
pat_mic
{ skull } git init
Initialized empty Git repository in C:/pat_mic/epn_code/proyectoGit/.git/
```

```
proyectoGit 17:56
proyectoGit git:(master) 17:56
```

Renombrar la rama master a **main**

```
pat_mic
{ skull } git branch -m main
```

```
proyectoGit git:(main) 18:01
```



Workshop – GIT STATUS



file.c - proyectoGit - Visual Studio Code

EXPLORER

OPEN EDITORS 1 unsaved

SSH FS - CONFIGURATIONS: CONFIGUR...

OUTLINE

PROYECTOGIT

- file.c U
- rama.dio U

file.c > ...

```
1 int main()
2 {
3     printf("hola git");
4     return 0;
5 }
6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS

pat_mic

```
( ) pwd
/c/pat_mic/epn_code/proyectoGit
pat_mic
( ) touch file.c
pat_mic
pat_mic
( ) git status
On branch main

No commits yet

Untracked files:
(use "git add <file>..." to include in what will be committed)
file.c
rama.dio

nothing added to commit but untracked files present (use "git add" to track)
pat_mic
( )
```



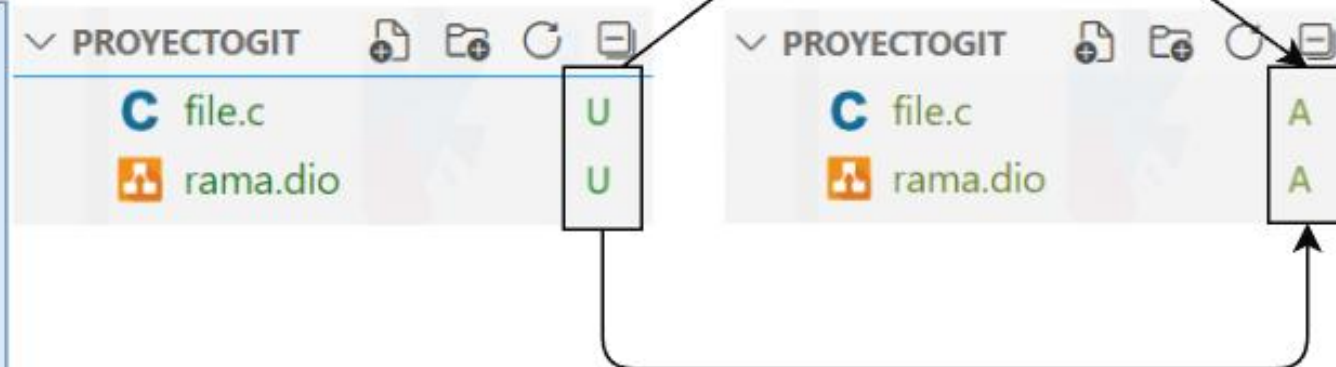
main
(RAMA/BRANCH)



Workshop – GIT ADD



MAIN



< 🦴 > git add file.c
< 🦴 > git add rama.dio

```
pat_mic  
< 🦴 > git status  
On branch main  
  
No commits yet  
  
Changes to be committed:  
  (use "git rm --cached <file>..." to unstage)  
    new file:   file.c  
    new file:   rama.dio
```



STAGE

MAIN

Work
Directory



```
PROYECTO GIT
file.c
rama.dio
```

```
pat_mic
$ git commit -m "Mis primero archivos al Stage"
[main (root-commit) 7fd395c] Mis primero archivos al Stage
2 files changed, 5 insertions(+)
create mode 100644 file.c
create mode 100644 rama.dio
```

```
PROYECTO GIT
file.c
rama.dio
```

```
$ git log
commit 7fd395cab3d4b2c062711cd77ced43ab8b94a11b (HEAD -> main)
Author: pat_mic <pat_mic@hotmail.com>
Date: Tue May 2 18:42:16 2023 -0500
```

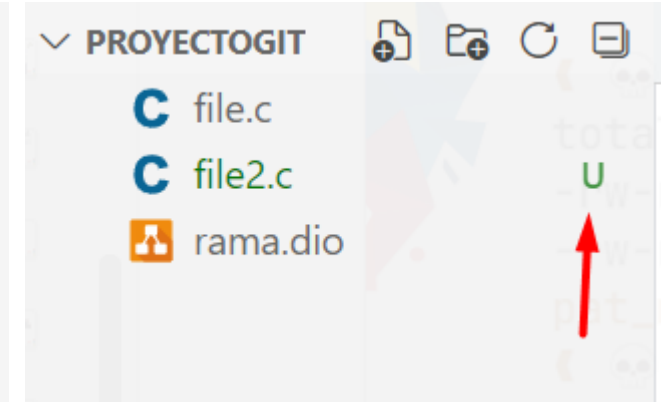
Mis primero archivos al Stage



Workshop - GIT

```
> ls -l
total 1
-rw-r--r-- 1 pat_m 197615 61 May  2 18:10 file.c
-rw-r--r-- 1 pat_m 197615  0 May  2 18:06 rama.dio
pat_mic
> touch file2.c
pat_mic
> ls -l
total 1
-rw-r--r-- 1 pat_m 197615 61 May  2 18:10 file.c
-rw-r--r-- 1 pat_m 197615  0 May  2 22:41 file2.c
-rw-r--r-- 1 pat_m 197615  0 May  2 18:06 rama.dio
pat_mic
> git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  file2.c

nothing added to commit but untracked files present (use "git add" to track)
```





Workshop – GIT log

```
pat_mic
[ 🦴 ] git add .
pat_mic
[ 🦴 ] git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   file2.c
```

```
pat_mic
[ 🦴 ] git commit -m "2do commit, file2.c al stage"
[main 255ddfff] 2do commit, file2.c al stage
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file2.c
```

```
pat_mic
[ 🦴 ] git add .
pat_mic
[ 🦴 ] git commit -m "3er commit, nuevo file3.c al stage"
[main 768545c] 3er commit, nuevo file3.c al stage
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file3.c
```

```
pat_mic
[ 🦴 ] git log
commit 768545c7da1995a91eaec1589853d6c2b67da750 (HEAD -> main)
Author: pat_mic <pat_mic@hotmail.com>
Date:   Tue May 2 22:49:19 2023 -0500
```

3er commit, nuevo file3.c al stage

```
commit 255ddfff430e537b7d3aa32bd1e9ba0b5081f6bd2
Author: pat_mic <pat_mic@hotmail.com>
Date:   Tue May 2 22:46:32 2023 -0500
```

2do commit, file2.c al stage

```
commit 7fd395cab3d4b2c062711cd77ced43ab8b94a11b
Author: pat_mic <pat_mic@hotmail.com>
Date:   Tue May 2 18:42:16 2023 -0500
```

Mis primero archivos al Stage





Workshop – cambiando contenido de los archivos



file3.c - proyectoGit - Visual Studio Code

EXPLORER

> OPEN EDITORS

> SSH FS - CONFIGURATIONS: CONF...

> OUTLINE

✓ PROYECTOGIT

file.c

file2.c

file3.c

rama.dio

M

M

file3.c

file3.c

You, 19 seconds ago | 1 author (You)

1 // este es un archivo con cambio You, now • Uncommitted changes

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

GITLENS

pat_mic

git status

On branch main

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: file3.c

modified: rama.dio

no changes added to commit (use "git add" and/or "git commit -a")

pat_mic

git status



Workshop – descartar cambios los cambios...



file3.c - proyectoGit - Visual Studio Code

EXPLORER

- > OPEN EDITORS
 - file3.c
- > SSH FS - CONFIGURATIONS: CONF...
- > OUTLINE
- ✓ PROYECTOGIT
 - file.c
 - file2.c
 - file3.c
 - rama.dio

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS

```
pat_mic  
( 🐼 ) git status  
On branch main  
Changes not staged for commit:  
  (use "git add <file>..." to update what will be committed)  
  (use "git restore <file>..." to discard changes in working directory)  
→ modified:   file3.c  
   modified:   rama.dio  
  
no changes added to commit (use "git add" and/or "git commit -a")  
pat_mic  
( 🐼 ) git checkout file3.c  
Updated 1 path from the index  
pat_mic  
( 🐼 )
```





Workshop – descartar todos los cambios

PROYECTOGIT

file.c

file2.c

file3.c

rama.dio

M

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

GITLENS

On branch main

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: file3.c

modified: rama.dio

no changes added to commit (use "git add" and/or "git commit -a")

pat_mic

git checkout file3.c

Updated 1 path from the index

pat_mic

git reset

Unstaged changes after reset:

M rama.dio

pat_mic

git status

On branch main

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: rama.dio

git log --graph

git log --graph --pretty=oneline

git log --graph --pretty=oneline --abbrev

git log --graph --decorate --oneline

PROYECTOGIT

file.c

file2.c

file3.c

rama.dio

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

GITLENS

```
git commit log --graph
* commit b4dde8108fc879bc9dec2cbcf9dcd4429330dabb (HEAD -> main)
  Author: pat_mic <pat_mic@hotmail.com>
  Date: Tue May 2 23:13:53 2023 -0500

    4to commit, todo ok

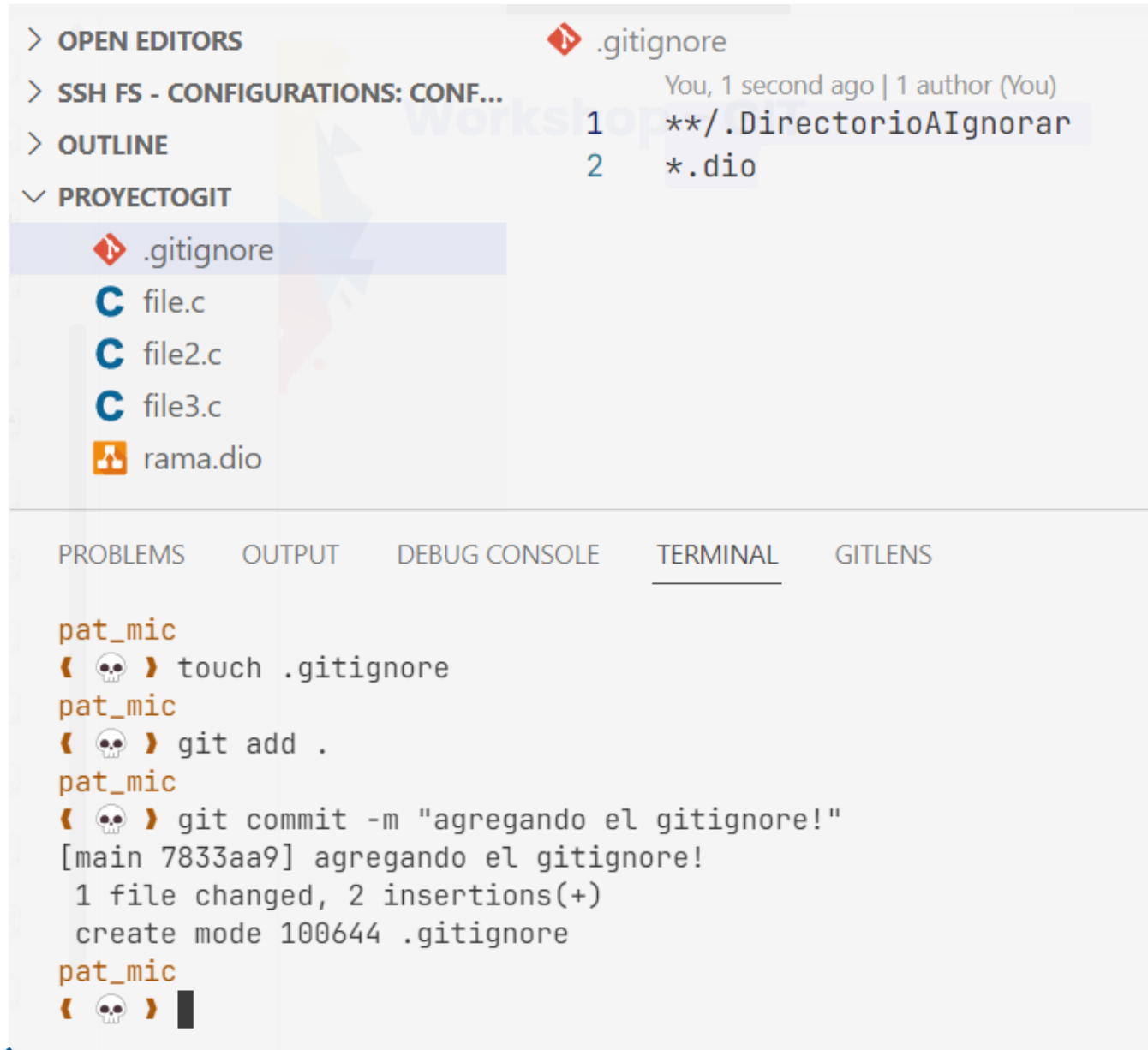
* commit 768545c7da1995a91eaec1589853d6c2b67da750
  Author: pat_mic <pat_mic@hotmail.com>
  Date: Tue May 2 22:49:19 2023 -0500

    3er commit, nuevo file3.c al stage

* commit 255ddff430e537b7d3aa32bd1e9ba0b5081f6bd2
  Author: pat_mic <pat_mic@hotmail.com>
  Date: Tue May 2 22:46:32 2023 -0500

    2do commit, file2.c al stage

* commit 7fd395cab3d4b2c062711cd77ced43ab8b94a11b
  Author: pat_mic <pat_mic@hotmail.com>
  Date: Tue May 2 18:42:16 2023 -0500
```



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar is open, showing a project named 'PROYECTO GIT' with a file tree containing '.gitignore', 'file.c', 'file2.c', 'file3.c', and 'rama.dio'. The '.gitignore' file is selected and its content is displayed in the editor: `1 **/.DirectorioAIgnorar` and `2 *.dio`. The file is attributed to 'You, 1 second ago | 1 author (You)'. Below the editor, the TERMINAL panel is active, showing a series of commands and their output. The commands are: `touch .gitignore`, `git add .`, and `git commit -m "agregando el gitignore!"`. The output shows the commit message and file changes: `[main 7833aa9] agregando el gitignore!`, `1 file changed, 2 insertions(+)`, and `create mode 100644 .gitignore`. The terminal prompt is `pat_mic`.

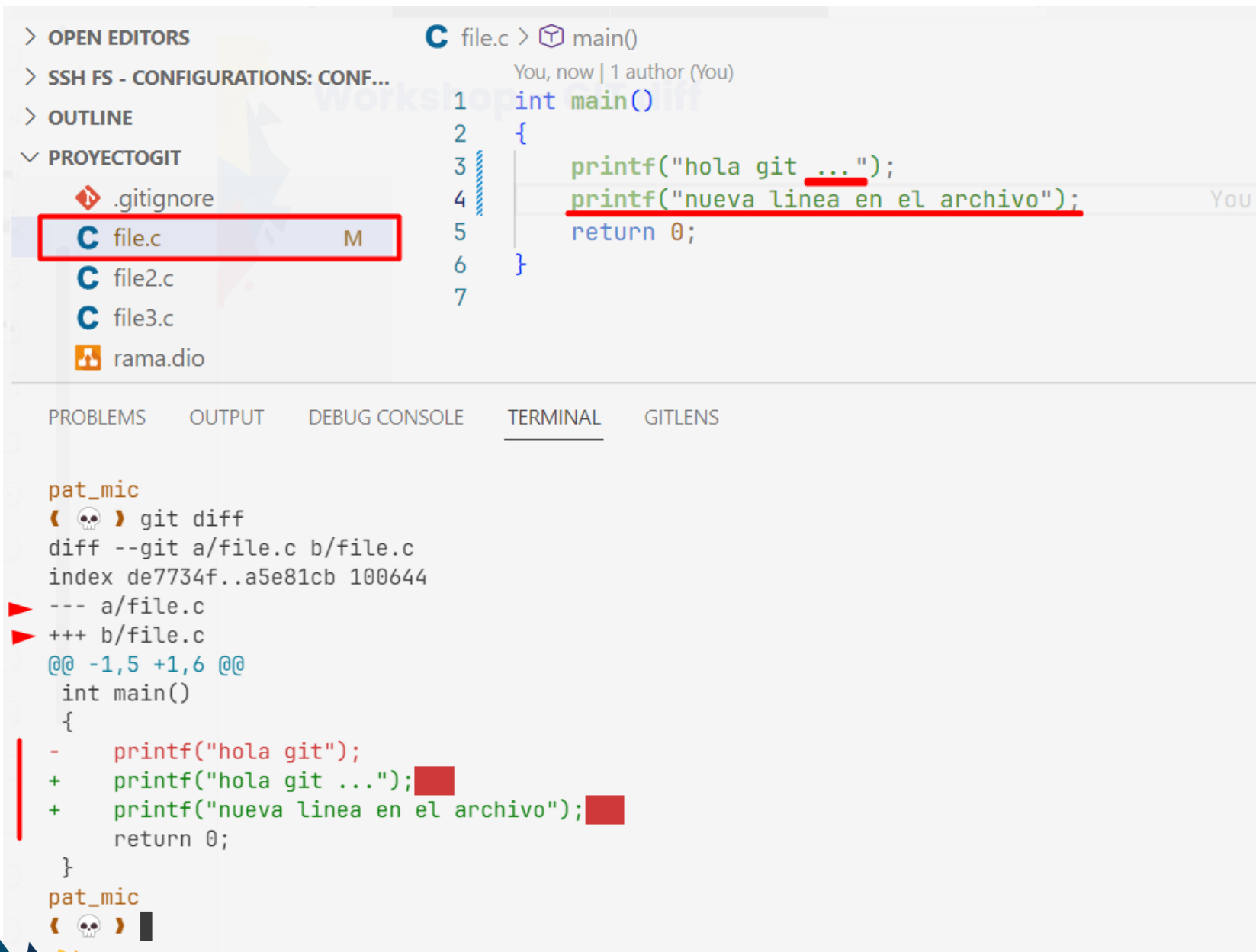
```
> OPEN EDITORS
> SSH FS - CONFIGURATIONS: CONF...
> OUTLINE
✓ PROYECTO GIT
  .gitignore
  file.c
  file2.c
  file3.c
  rama.dio

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  GITLENS

pat_mic
{ skull } touch .gitignore
pat_mic
{ skull } git add .
pat_mic
{ skull } git commit -m "agregando el gitignore!"
[main 7833aa9] agregando el gitignore!
1 file changed, 2 insertions(+)
create mode 100644 .gitignore
pat_mic
{ skull }
```



Workshop – GIT diff



```
> OPEN EDITORS
> SSH FS - CONFIGURATIONS: CONF...
> OUTLINE
✓ PROYECTOGIT
  .gitignore
  C file.c M
  C file2.c
  C file3.c
  rama.dio

C file.c > main()
You, now | 1 author (You)
1 int main()
2 {
3     printf("hola git ...");
4     printf("nueva linea en el archivo");
5     return 0;
6 }
7

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS

pat_mic
( ) git diff
diff --git a/file.c b/file.c
index de7734f..a5e81cb 100644
--- a/file.c
+++ b/file.c
@@ -1,5 +1,6 @@
 int main()
 {
-    printf("hola git");
+    printf("hola git ...");
+    printf("nueva linea en el archivo");
     return 0;
 }
pat_mic
( )
```

Workshop – GIT checkout

```
pat_mic
( skull ) git log --decorate --oneline
edcc638 (HEAD -> main) nuevo commit
7833aa9 agregando el gitignore!
b4dde81 4to commit, todo ok
768545c 3er commit, nuevo file3.c al stage
255ddff 2do commit, file2.c al stage
7fd395c Mis primero archivos al Stage
pat_mic
( skull ) git checkout 768545c
Note: switching to '768545c'.
```

PROYECTO GIT

- .gitignore
- file.c
- file2.c
- file3.c
- rama.dio

```
pat_mic
( skull ) git log git log --decorate --oneline
768545c (HEAD) 3er commit, nuevo file3.c al stage
255ddff 2do commit, file2.c al stage
7fd395c Mis primero archivos al Stage
pat_mic
( skull )
```

PROYECTO GIT

- file.c
- file2.c
- file3.c
- rama.dio





Workshop – GIT reset – – hard



pat_mic

pat_mic
[skull] git log --decorate --oneline

edcc638 (HEAD, main) nuevo commit

7833aa9 agregando el gitignore!

b4dde81 4to commit, todo ok

768545c 3er commit, nuevo file3.c al stage

255ddff 2do commit, file2.c al stage

7fd395c Mis primero archivos al Stage

pat_mic

[skull] git reset --hard 768545c

HEAD is now at 768545c 3er commit, nuevo file3.c al stage

pat_mic

[skull] git log --decorate --oneline

768545c (HEAD) 3er commit, nuevo file3.c al stage

255ddff 2do commit, file2.c al stage

7fd395c Mis primero archivos al Stage

pat_mic

[skull]

pat_mic

[skull] git reflog

768545c (HEAD) HEAD@{0}: reset: moving to 768545c

edcc638 (main) HEAD@{1}: checkout: moving from 768545c7da1995a91eaec1

768545c (HEAD) HEAD@{2}: checkout: moving from main to 768545c

edcc638 (main) HEAD@{3}: commit: nuevo commit

7833aa9 HEAD@{4}: commit: agregando el gitignore!

b4dde81 HEAD@{5}: commit: 4to commit, todo ok

768545c (HEAD) HEAD@{6}: reset: moving to HEAD

768545c (HEAD) HEAD@{7}: commit: 3er commit, nuevo file3.c al stage

255ddff HEAD@{8}: commit: 2do commit, file2.c al stage

7fd395c HEAD@{9}: commit (initial): Mis primero archivos al Stage

pat_mic



Workshop - GIT



pat_mic

```
( 🦴 ) git log --decheckout edcc638  
Previous HEAD position was 768545c 3er commit, nuevo file3.c al stage  
HEAD is now at edcc638 nuevo commit
```

pat_mic

```
( 🦴 ) git log --decorate --oneline  
edcc638 (HEAD, main) nuevo commit  
7833aa9 agregando el gitignore!  
b4dde81 4to commit, todo ok  
768545c 3er commit, nuevo file3.c al stage  
255ddff 2do commit, file2.c al stage  
7fd395c Mis primero archivos al Stage
```

YO RESUELVO
MIS PROBLEMAS
CON CALMA ...



pat_mic

```
( 🦴 ) git reset --hard edcc638  
HEAD is now at edcc638 nuevo commit
```

pat_mic

```
( 🦴 ) git log --decorate --oneline  
edcc638 (HEAD, main) nuevo commit  
7833aa9 agregando el gitignore!  
b4dde81 4to commit, todo ok  
768545c 3er commit, nuevo file3.c al stage  
255ddff 2do commit, file2.c al stage  
7fd395c Mis primero archivos al Stage
```

pat_mic

```
( 🦴 ) █
```



Workshop – GIT tag NOMBRE_MARCA

```
pat_mic
[ skull ] git log --decorate --oneline
edcc638 (HEAD, main) nuevo commit
7833aa9 agregando el gitignore!
b4dde81 4to commit, todo ok
768545c 3er commit, nuevo file3.c al stage
255ddff 2do commit, file2.c al stage
7fd395c Mis primero archivos al Stage

pat_mic
[ skull ] git tag ver1.0

pat_mic
[ skull ] git log --decorate --oneline
edcc638 (HEAD, tag: ver1.0, main) nuevo commit
7833aa9 agregando el gitignore!
b4dde81 4to commit, todo ok
768545c 3er commit, nuevo file3.c al stage
255ddff 2do commit, file2.c al stage
7fd395c Mis primero archivos al Stage

pat_mic
[ skull ] touch fileX.c

pat_mic
pat_mic
[ skull ] git add .

pat_mic
[ skull ] git commit -m "Nuevo commit luego de un tag"
[detached HEAD 41b815d] Nuevo commit luego de un tag
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 fileX.c
```

```
[ skull ] git commit -m "Nuevo commit luego de un tag"
[detached HEAD 41b815d] Nuevo commit luego de un tag
41b815d (HEAD) Nuevo commit luego de un tag
edcc638 (tag: ver1.0, main) nuevo commit
7833aa9 agregando el gitignore!
b4dde81 4to commit, todo ok
768545c 3er commit, nuevo file3.c al stage
255ddff 2do commit, file2.c al stage
7fd395c Mis primero archivos al Stage

pat_mic
[ skull ] git checkout ver1.0
Warning: you are leaving 1 commit behind, not connected to
any of your branches:

41b815d Nuevo commit luego de un tag

If you want to keep it by creating a new branch, this may be a goo
to do so with:

git branch <new-branch-name> 41b815d

HEAD is now at edcc638 nuevo commit

pat_mic
[ skull ] git log --decorate --oneline
edcc638 (HEAD, tag: ver1.0, main) nuevo commit
7833aa9 agregando el gitignore!
b4dde81 4to commit, todo ok
768545c 3er commit, nuevo file3.c al stage
255ddff 2do commit, file2.c al stage
7fd395c Mis primero archivos al Stage
```



Workshop – GIT branch



```
pat_mic                                proyectoGit git:(main) ⚡ 14:51
pat_mic ❷ git branch RamaLogin
pat_mic                                proyectoGit git:(main) ⚡ 14:52
pat_mic ❷ git switch RamaLogin
Switched to branch 'RamaLogin'
pat_mic                                proyectoGit git:(RamaLogin) ⚡ 14:52
pat_mic ❷ git log --decorate --oneline
edcc638 (HEAD -> RamaLogin, tag: ver1.0, main) nuevo commit
7833aa9 agregando el gitignore
b4dde81 4to commit, todo ok
768545c 3er commit, nuevo file3.c al stage
255ddff 2do commit, file2.c al stage
7fd395c Mis primero archivos al Stage
pat_mic                                proyectoGit git:(RamaLogin) ⚡ 14:52
pat_mic ❷ touch login.c
pat_mic                                proyectoGit git:(RamaLogin) ⚡ 14:53
pat_mic ❷ git add .
pat_mic                                proyectoGit git:(RamaLogin) ⚡ 14:53
pat_mic ❷ git commit -m "1er commit en la ramaLogin"
[RamaLogin ef29dcc] 1er commit en la ramaLogin
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 login.c
```



Workshop – GIT branch



SOURCE CONTROL ...

login.c ×

login.c > main()

You, 1 second ago | 1 author (You)

```
1 int main()
2 {
3     printf("Funcionalidad del Login OK");
4     return 0;
5 }
6
```

BRANCHES (2)

- > RamaLogin 41 seconds ago ✓
- > main 15 hours ago

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** GITLENS

bash + v [] [] ... ×

pat_mic

```
( ) git commit -m "Funcionalidad Login OK"
[RamaLogin 6a98dd7] Funcionalidad Login OK
1 file changed, 5 insertions(+)
pat_mic
pat_mic
( ) ls -l
total 154
-rw-r--r-- 1 pat_m 197615 110 May 3 00:45 file.c
-rw-r--r-- 1 pat_m 197615 0 May 2 22:41 file2.c
-rw-r--r-- 1 pat_m 197615 0 May 3 14:30 file3.c
-rw-r--r-- 1 pat_m 197615 76 May 3 14:59 login.c
-rw-r--r-- 1 pat_m 197615 155420 May 3 00:45 rama.dio
```

proyectoGit git:(main) 14:51

proyectoGit git:(main) 14:52

proyectoGit git:(RamaLogin) 14:52

proyectoGit git:(RamaLogin) 14:59

proyectoGit git:(RamaLogin) 15:00

proyectoGit git:(RamaLogin) 15:00

proyectoGit git:(RamaLogin) 14:53



Workshop – GIT merge



```
pat_mic
( 🦴 ) git switch main
Switched to branch 'main'
pat_mic
( 🦴 ) git merge RamaLogin
Updating edcc638..6a98dd7
Fast-forward
 login.c | 5 +++++
 1 file changed, 5 insertions(+)
 create mode 100644 login.c
pat_mic
( 🦴 ) ls
file.c  file2.c  file3.c  login.c  rama.dio
```

```
🦴 proyectoGit git:(RamaLogin) ⚡
```

```
🦴 proyectoGit git:(main) ⚡
```

```
🦴 proyectoGit git:(main) ⚡
```

```
pat_mic
( 🦴 ) git log git log --decorate --oneline
6a98dd7 (HEAD -> main, RamaLogin) Funcionalidad Login OK
ef29dcc 1er commit en la ramaLogin
edcc638 (tag: ver1.0) nuevo commit
7833aa9 agregando el gitignore!
b4dde81 4to commit, todo ok
768545c 3er commit, nuevo file3.c al stage
255ddff 2do commit, file2.c al stage
7fd395c Mis primero archivos al Stage
```

```
🦴 proyectoGit git:(main) ⚡
```



Workshop – GIT merge

EXPLORER

- PROJECTOGIT
 - .gitignore
 - file.c
 - file2.c
 - file3.c
 - login.c 3, !
 - rama.dio

login.c 3, !

```
3 <<<<<< HEAD (Current Change)
4     printf("Funcionalidad del Login OK...");
5     =====
6     printf("Funcionalidad del Login OK, falta el loguot");
7     printf("Modificacion realizada desde la rama main ??????..
8 >>>>>> main (Incoming Change)
9     return 0;
```

Resolve in Merge Editor

PROBLEMS 3

pat_mic

```
> git log --decorate --oneline
ce7afd1 (HEAD -> RamaLogin) antes del merge
6a98dd7 Funcionalidad Login OK
ef29dcc 1er commit en la ramaLogin
edcc638 (tag: ver1.0) nuevo commit
7833aa9 agregando el gitignore!
b4dde81 4to commit, todo ok
768545c 3er commit, nuevo file3.c al stage
255ddff 2do commit, file2.c al stage
7fd395c Mis primero archivos al Stage
```

pat_mic

```
> git merge main
```

Auto-merging login.c
CONFLICT (content): Merge conflict in login.c
Automatic merge failed; fix conflicts and then commit the result.

projectoGit git:(RamaLogin) 15:35

projectoGit git:(RamaLogin) 15:36



Workshop – GIT merge

```
C login.c 3,! X
C login.c > main()
You, 15 seconds ago | 1 author (You)
1 int main()
2 {
3 <<<<<< HEAD (Current Change)
4     printf("Funcionalidad del Login OK...");
5
6 >>>>>>
7     printf("Funcionalidad del Login OK, falta el loguot");
8     printf("Modificacion realizada desde la rama main ??????..!");
9 >>>>>> main (Incoming Change)
10     return 0;
11 }
```



C login.c ! • C login.c: Current Changes ↔ Incoming Changes C Merging: login.c ! •

C login.c > main()

Incoming 59e9a71 • main

Accept Incoming | Accept Combination (Incoming First) | Ignore

```
3 printf("Funcionalidad del Login OK, falta el loguot");
4 printf("Modificacion realizada desde la rama main ??????..!");
5 return 0;
6 }
```

Current ce7afd1 • RamaLogin

Accept Current | Accept Combination (Current First) | Ignore

```
3 printf("Funcionalidad del Login OK...");
4
5 return 0;
6 }
```

Result login.c

No Changes Accepted

```
3 printf("Funcionalidad del Login OK");
4 return 0;
5 }
6 }
```

1 Conflict Remaining

Complete Merge



Workshop – GIT diff

pat_mic

```
pat_mic
[ 🦴 ] git switch main
fatal: cannot switch branch while merging
Consider "git merge --quit" or "git worktree add".
```

pat_mic

```
pat_mic
[ 🦴 ] git diff RamaLogin
diff --git a/login.c b/login.c
index 91d45fa..d61380c 100644
--- a/login.c
+++ b/login.c
@@ -1,5 +1,6 @@
 int main()
 {
-    printf("Funcionalidad del Login OK...");
+    printf("Funcionalidad del Login OK, falta el loguot");
+    printf("Modificacion realizada agregamos el logout");
     return 0;
 }
```

pat_mic

```
pat_mic
[ 🦴 ] git merge RamaLogin
fatal: You have not concluded your merge (MERGE_HEAD exists).
Please, commit your changes before you merge.
```

pat_mic

```
pat_mic
[ 🦴 ] git commit -m "merge"
[RamaLogin 07d3250] merge
```





Workshop – GIT Branch –d Nombre Rama

pat_mic

`git branch`

* **RamaLogin**

main

pat_mic

`git branch -g RamaLogin`

error: unknown switch `g'

usage: git branch [<options>] [-r | -a] [--merged] [--no-merged]

or: git branch [<options>] [-f] [--recurse-submodules] <branch-name> [<start-point>]

or: git branch [<options>] [-l] [<pattern>...]

proyectoGit git:(**RamaLogin**) 23:59

proyectoGit git:(**RamaLogin**) 23:59

pat_mic

`git checkout main`

Switched to branch 'main'

pat_mic

`git branch -g RamaLogin`

proyectoGit git:(**RamaLogin**) 23:59

proyectoGit git:(**main**) 00:00



training.github.com/downloads/es_ES/github-git-cheat-sheet.pdf

SINCRONIZAR CAMBIOS

Registrar un marcador de repositorio e intercambiar historial de versión

```
$ git fetch [bookmark]
```

Descarga todo el historial del marcador del repositorio

```
$ git merge [bookmark]/[branch]
```

Combina la rama del marcador con la rama local actual

```
$ git push [alias] [branch]
```

Carga todos los commits de la rama local al GitHub

```
$ git pull
```

Descarga el historial del marcador e incorpora cambios



Sign in to GitHub

Username or email address

Password

[Forgot password?](#)

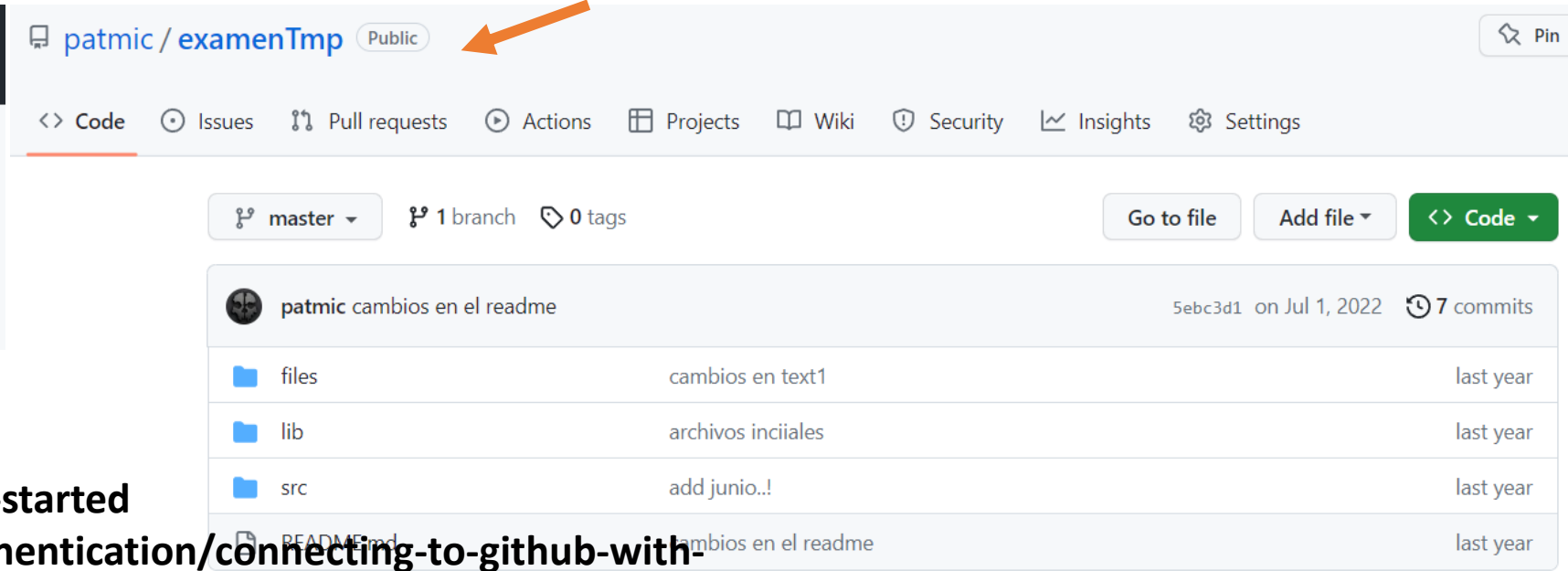
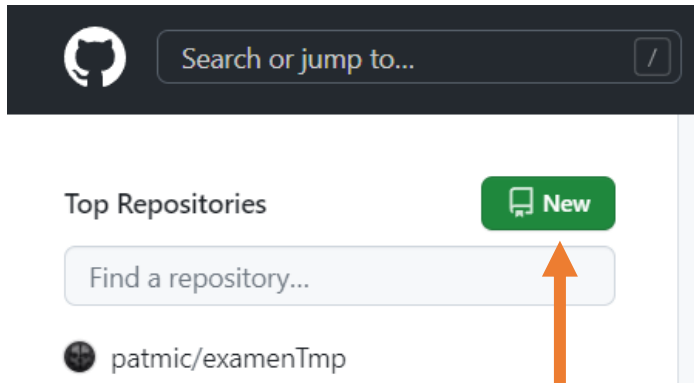
Sign in

New to GitHub? [Create an account.](#)





Workshop – GitHub



<https://docs.github.com/es/get-started>

<https://docs.github.com/es/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent>

```
pat_mic
{ 🦴 } cd ~
pat_mic
{ 🦴 } pwd
/c/Users/pat_m
pat_mic
{ 🦴 } mkdir .ssh
pat_mic
{ 🦴 } cd .ssh
pat_mic
{ 🦴 } pwd
/c/Users/pat_m/.ssh
```



Workshop – GitHub

- 1 Abra Git Bash.
- 2 Pega el siguiente texto, que sustituye tu dirección de correo electrónico en GitHub.

```
$ ssh-keygen -t ed25519 -C "your_email@example.com"
```

pat_mic

```
( skull ) ssh-keygen -t ed25519 -C "pat_mic@hotmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/pat_m/.ssh/id_ed25519): pat_rsa
pat_rsa already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in pat_rsa
Your public key has been saved in pat_rsa.pub
The key fingerprint is:
SHA256:tEgPzaSgdewMzl0XL+cRn8P6ZC6IUXgZK0hbyvs4Cak pat_mic@hotmail.com
The key's randomart image is:
+--[ED25519 256]--+
|    00. ..0..    |
|   0000=0..+ + .  |
|  .+ =+0=. = + =  |
|   +.== + + 0 .   |
|  0 +. S   0 0    |
|  0 +   0 . =     |
|  . . 0   . . . 0  |
|E  +.           .  |
|   .0.          |
+-----[SHA256]-----+
```

dar enter



Workshop – GitHub

- 1 Verifica que el agente ssh-agent se esté ejecutando. Puedes usar las instrucciones de "Auto-lanzamiento ssh-agent" en "[Trabajar con contraseñas de clave SSH](#)" o iniciarla manualmente:

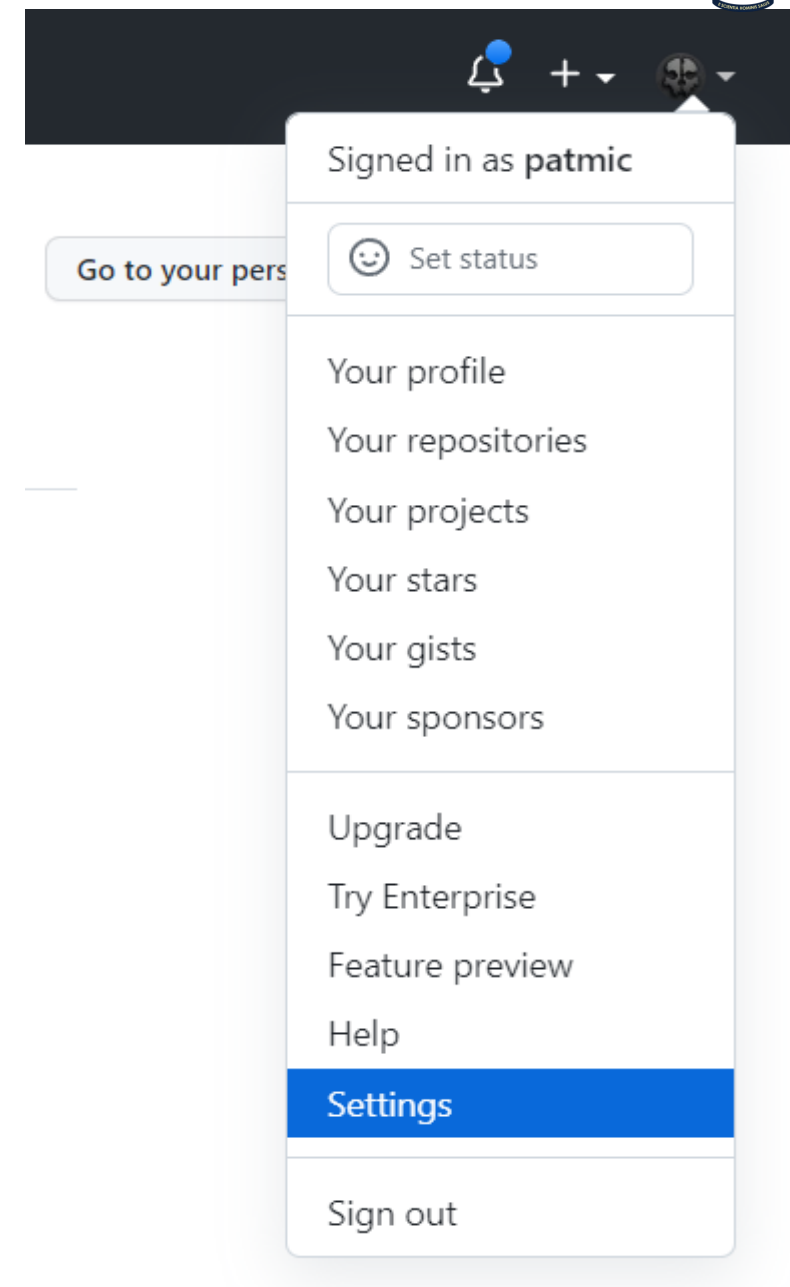
```
# start the ssh-agent in the background
$ eval "$(ssh-agent -s)"
> Agent pid 59566
```

- 2 Agrega tu llave privada SSH al ssh-agent. Si ha creado su clave con un nombre diferente o si está agregando una clave existente que tenga un nombre diferente, reemplaza `id_ed25519` en el comando con el nombre de su archivo de clave privada.

```
$ ssh-add ~/.ssh/id_ed25519
```

- 3 Agrega la llave SSH a tu cuenta en GitHub. Para obtener más información, vea «[Agregar una clave SSH nueva a tu cuenta de GitHub](#)».

```
pat_mic
❯ ls
pat_rsa  pat_rsa.pub
pat_mic
❯ eval "$(eval "$(ssh-agent -s)"
Agent pid 722
pat_mic
❯ ssh-add ~/.ssh/id_ed25519
/c/Users/pat_m/.ssh/id_ed25519: No such file or directory
pat_mic
❯ ssh-add ~/.ssh/pat_rsa
Identity added: /c/Users/pat_m/.ssh/pat_rsa (pat_mic@hotmail.com)
```





pat_mic (patmic)

Your personal account

Go

Public profile

Account

Appearance

Accessibility

Notifications

Access

Billing and plans

Emails

Password and authentication

Sessions

SSH and GPG keys

Organizations

Moderation

SSH keys

New SSH key

There are no SSH keys associated with your account.

Check out our guide to [generating SSH keys](#) or troubleshoot [common SSH problems](#).

GPG keys

New GPG key

There are no GPG keys associated with your account.

Learn how to [generate a GPG key and add it to your account](#).

Vigilant mode

☐ Flag unsigned commits as unverified

This will include any commit attributed to your account but not signed with your GPG or S/MIME key.

Note that this will include your existing unsigned commits.



```
❗ cat pat_rsa.pub  
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIBmJHQrGH9MdOx0GYy4r7bIJWBDI62KI5RC4YqeXeX2G pat_mic@hotmail.com
```

SSH keys / Add new

Title

pat_repo

Key type

Authentication Key

Key

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIBmJHQrGH9MdOx0GYy4r7bIJWBDI62KI5RC4YqeXeX2G  
pat_mic@hotmail.com
```



Add SSH key



← All products

Authentication

- > Seguridad de la cuenta
- > Proteger tu cuenta con 2FA
- ✓ Conexión con SSH
 - Acerca de SSH
 - Reenvío del agente SSH
 - Administrar las llaves de despliegue
 - Comprobación de la clave SSH existente
 - Generación de una clave SSH nueva
 - Incorporación de una clave SSH nueva
 - Prueba de la conexión SSH**
 - Frases de contraseña de clave SSH
- > Solucionar problemas de SSH
- > Verificar las firmas de confirmación
- > Solucionar los problemas de la verificación

Mac Windows Linux

```
pat_mic  
[ 🐼 ] ssh -T git@github.com  
The authenticity of host 'github.com (140.82.113.4)' can't be established.  
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvC0qU.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.  
Hi patmic! You've successfully authenticated, but GitHub does not provide shell access.
```

- 1 Abra Git Bash.
- 2 Escriba lo siguiente:

```
$ ssh -T git@github.com  
# Attempts to ssh to GitHub
```

Puedes ver una advertencia como la siguiente:

```
> The authenticity of host 'github.com (IP ADDRESS)' can't be established.  
> ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvC0qU.  
> Are you sure you want to continue connecting (yes/no)?
```



Workshop – GitHub

```
pat_mic > cd C:/pat_mic/epn_code/proyectoGit
pat_mic > pwd
/c/pat_mic/epn_code/proyectoGit
```

```
pat_mic > git remote add origin git@github.com:patmic/pat_git.git
pat_mic > git status
On branch main
nothing to commit, working tree clean
pat_mic > git push -u origin main
Enumerating objects: 25, done.
Counting objects: 100% (25/25), done.
Delta compression using up to 16 threads
Compressing objects: 100% (23/23), done.
Writing objects: 100% (25/25), 111.80 KiB | 1.93 MiB/s, done.
Total 25 (delta 7), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (7/7), done.
To github.com:patmic/pat_git.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```



Workshop – GitHub

pat_git / README.md in main

<> Edit new file

👁 Preview

```
1 # pat_git
2 se agrega el readme al proyecto
```

solo trae el historial

```
❗ git fetch
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 687 bytes | 85.00 KiB/s, done.
From github.com:patmic/pat_git
 59e9a71..ef87697  main    -> origin/main
```

▼ PROYECTOGIT

- 🔴 .gitignore
- 🔵 file.c
- 🔵 file2.c
- 🔵 file3.c
- 🔵 login.c
- 🔴 rama.dio

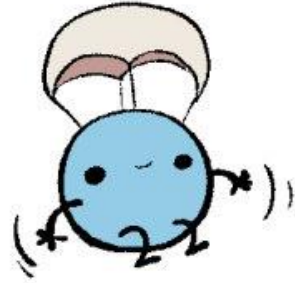
pat_mic

```
❗ git pull
Updating 59e9a71..ef87697
Fast-forward
 README.md | 2 ++
 1 file changed, 2 insertions(+)
 create mode 100644 README.md
```

Trae el historial y los cambios

▼ PROYECTOGIT

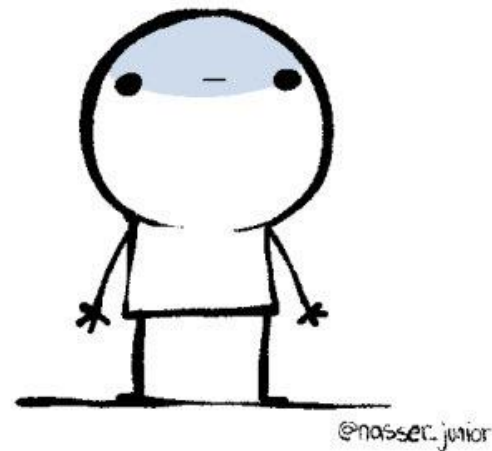
- 🔴 .gitignore
- 🔵 file.c
- 🔵 file2.c
- 🔵 file3.c
- 🔵 login.c
- 🔴 rama.dio
- 🔴 README.md



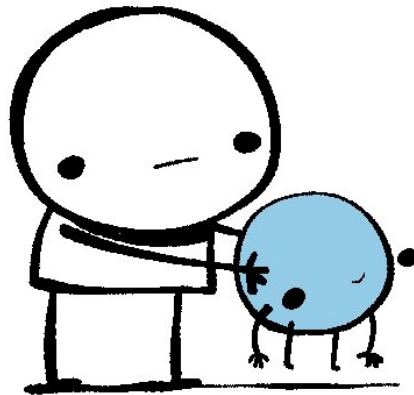
Git pull



--force



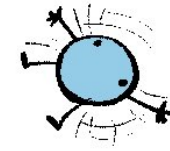
Git push



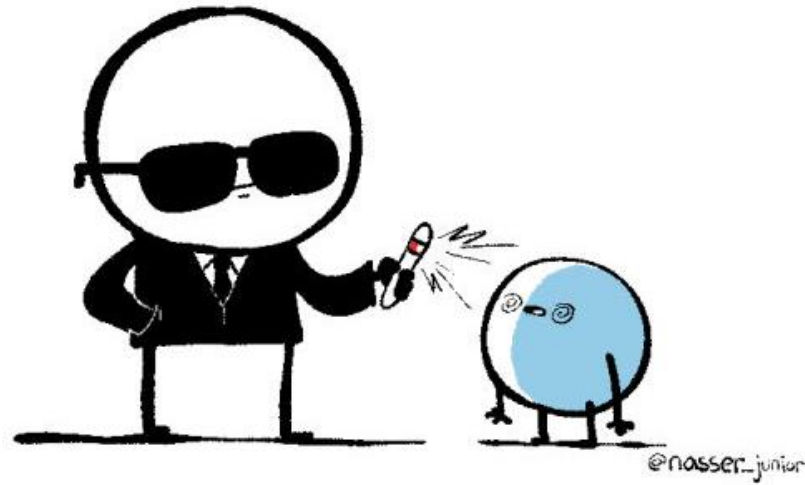
-force



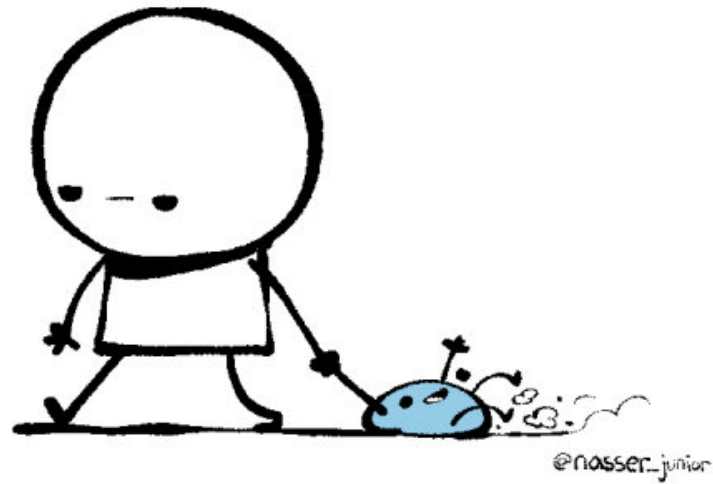
@naeser_junior



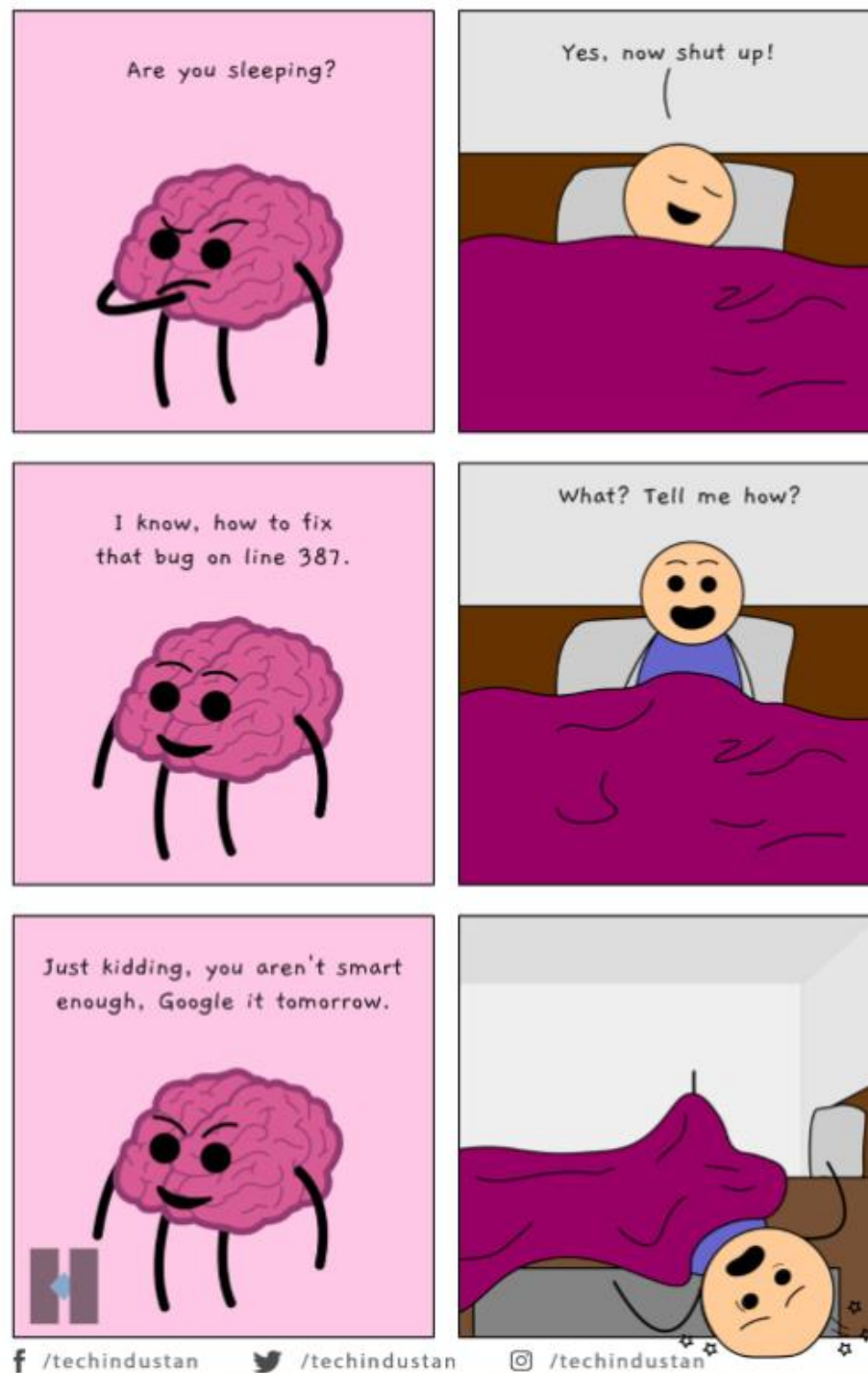
Git reset --hard

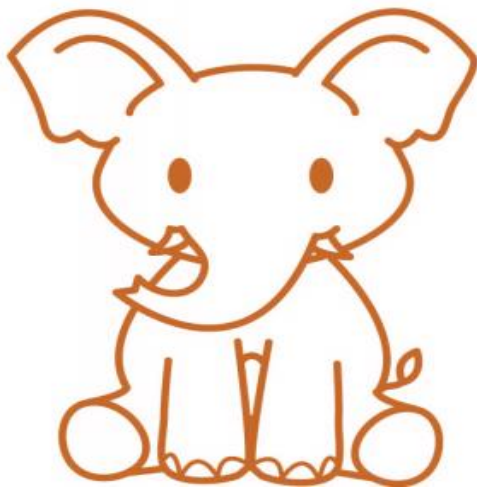


Git revert









TROMPUD@



chuki

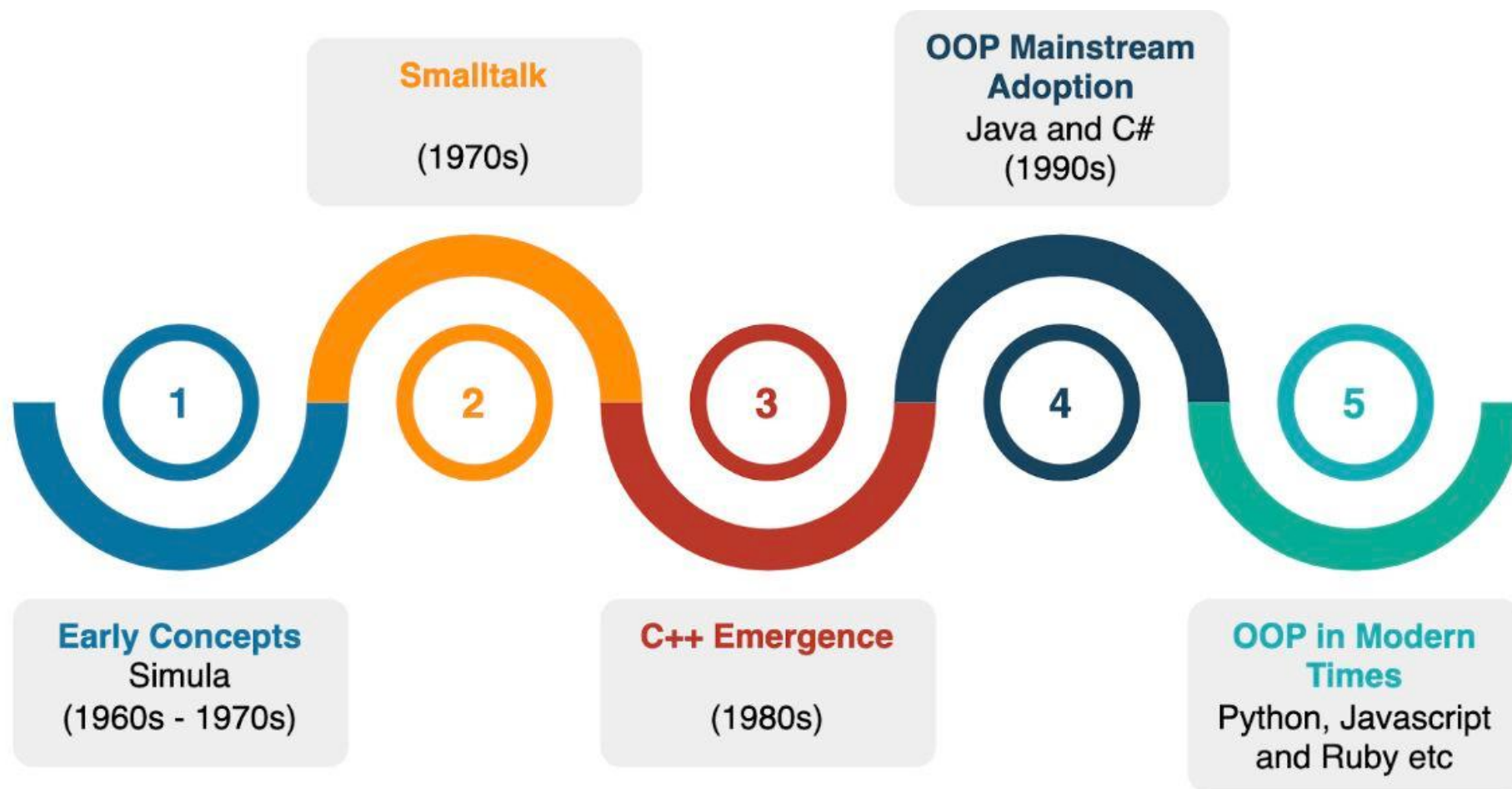


< toxic../>





Object-oriented Programming: A Primer



Timeline of OOP adoption





Origen de JAVA



Marc Andreessen y Jim Clark

DE OAK (DE PAGO)
A JAVA (GRATIS)

Grandes empresas que confían en Java



The infographic shows a central Java logo with lines connecting it to the logos of several major companies that use Java:

- NASA World Wind
- Minecraft
- Twitter
- Uber
- Netflix
- Ebay

¿CÓMO FUNCIONA JAVA?



TIPOS DE LENGUAJE

COMPILADO



Convierte el código a binarios que lee el sistema operativo.



INTERPRETADO



Requieren de un programa que lea la instrucción del código en tiempo real, y la ejecute.



INTERMEDIO



Se compila el código fuente a un lenguaje intermedio y este último se ejecuta en una máquina virtual





OpenJDK





Java Micro Edition



Java Standard Edition



Java Enterprise Edition



JAVA SWING

JavaFx™



It is used for:

- Mobile applications (specially Android apps)
- Desktop applications
- Web applications
- Web servers and application servers
- Games
- Database connection
- And much, much more!





Why Use Java?

- Java works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)
- It is one of the most popular programming language in the world
- It has a large demand in the current job market
- It is easy to learn and simple to use
- It is open-source and free
- It is secure, fast and powerful
- It has a huge community support (tens of millions of developers)
- Java is an **object oriented** language which gives a clear structure to programs and allows code to be reused, lowering development costs
- As Java is close to [C++](#) and [C#](#), it makes it easy for programmers to switch to Java or vice versa

COMPILACIÓN

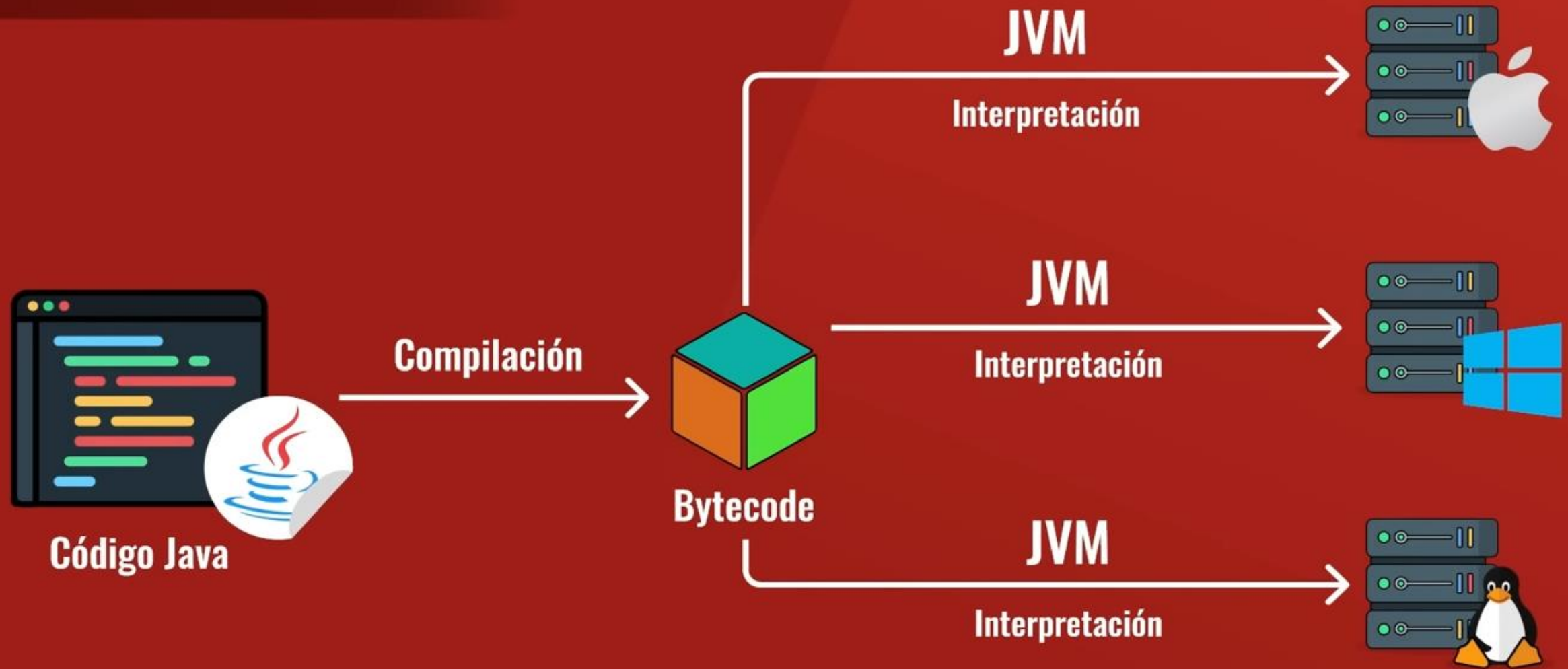


INTERPRETACIÓN



Ejecución de los aplicativos de java

Compilación Just In Time





Todos los archivos
**pertenecen a un
paquete**

Importa los paquetes
para el proyecto

Java usa clases para
ejecutar el código

Se debe indicar el
tipo de dato

Modificadores de acceso:
private, public, protected
o por defecto ninguno

El método principal
en Java es **el método**

La palabra reservada **new**
**crea un objeto del tipo
de dato especificado**

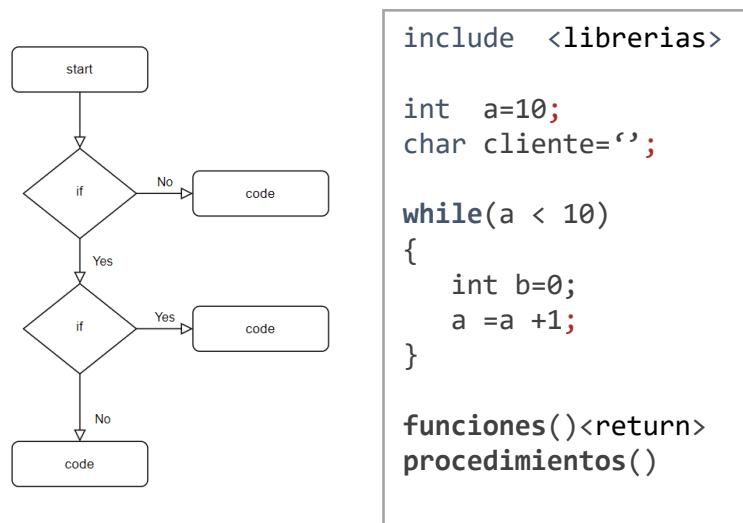
```
1 package team.ed.course;
2 import java.lang.*;
3 public class Person {
4     private String name;
5     public static void main(String args[]){
6         Person friend = new Person();
7         friend.name = "Peter";
8         System.out.println("Hola " +
9             friend.name);
10    }
11 }
```

Se utilizan **;**
para **cada sentencia**

Se usan **{ }** para identificar el
bloque de código



diseño del programa + realista (UML)



Programación estructurada



tipos de datos

- int
- float
- char
- bool
- ...

control de flujo

- if - else
- for(n)
- while(<> !=)
- do (<> != ==)
- beak
- switch



```
import <paquetes>

//propiedades, atributos
Public int saldo=1;
Private char cliente='';
String nombre="";

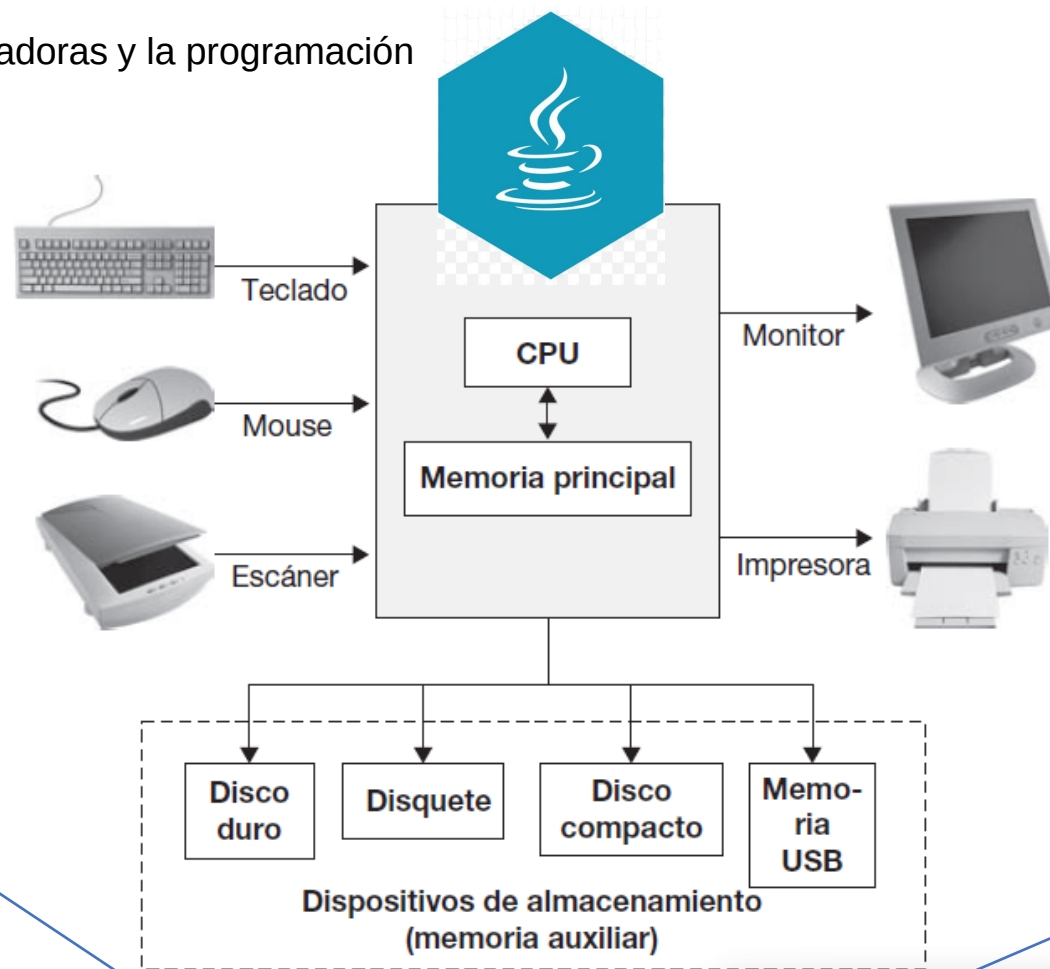
while(a < 10)
{
    int a =a +1;
}

métodos()<return>
"funciones"
```

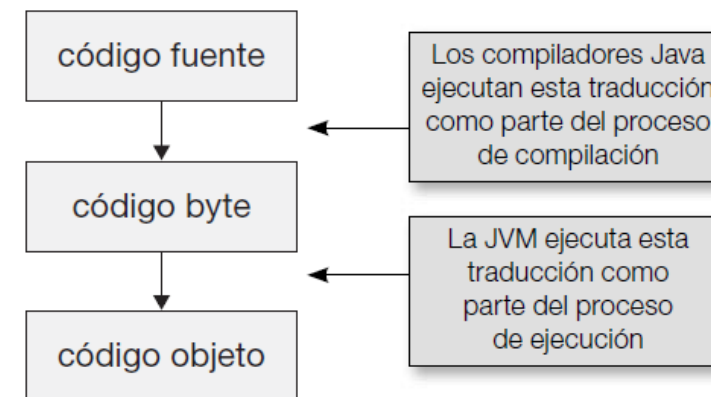
Programación
Orientada **Objetos**



Introducción a las computadoras y la programación



Portabilidad



Winx + JVM

10101000010001

Linux + JVM

10101111110001

O.S + JVM

00001111110001

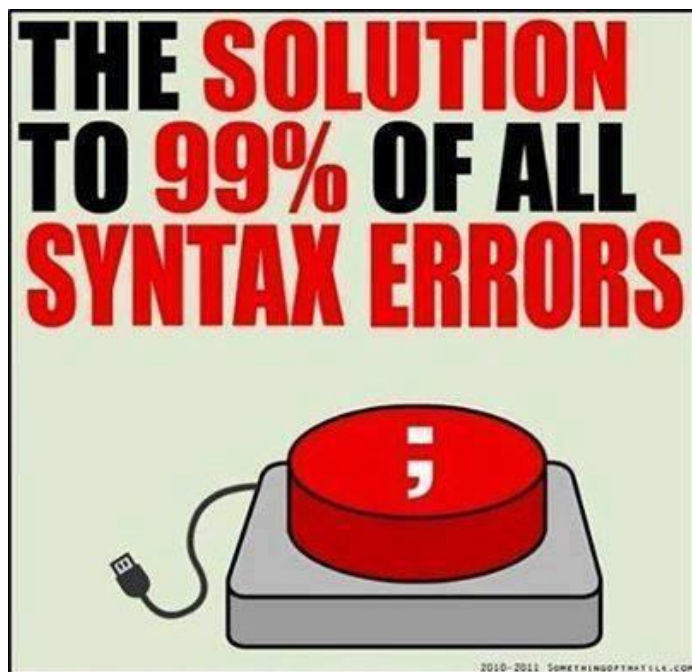
101class0010int=001xxxx
.class

```
import <paquetes>

//propiedades, atributos
Public int saldo=1;
Private char cliente='';
String nombre="";

while(a < 10)
{
    a =a +1;
}

métodos()<return>
"funciones"
```





Language Ranking: IEEE Spectrum

Rank	Language	Type	Score
1	Python▼	  	100.0
2	Java▼	  	95.3
3	C▼	  	94.6
4	C++▼	  	87.0
5	JavaScript▼		79.5
6	R▼		78.6
7	Arduino▼		73.2
8	Go▼	 	73.1
9	Swift▼	 	70.5
10	Matlab▼		68.4



1. Problema

2. Solución

- Algoritmo (pseudocódigo)
- Diagrama de flujo
- Código (debugging)
- **Traza** (Trace) – prueba de escritorio

Variables : 40 , 10, 4

Largo = 10 ancho = 4

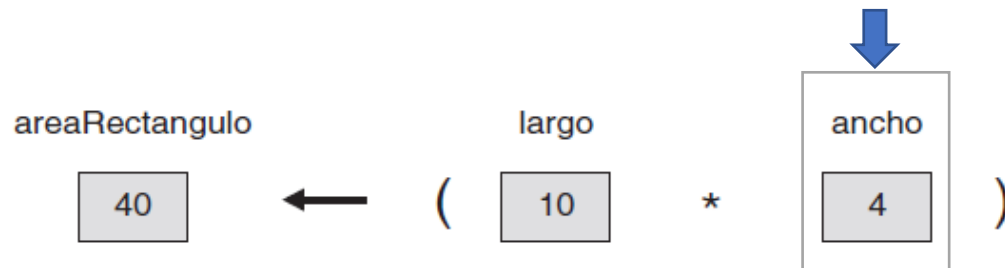
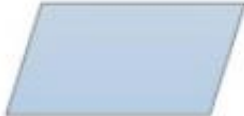
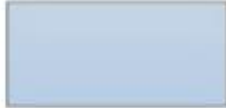


Diagrama de flujo

Símbolo	Nombre	Función
	Inicio / Final	Representa el inicio y el final de un proceso
	Linea de Flujo	Indica el orden de la ejecución de las operaciones. La flecha indica la siguiente instrucción.
	Entrada / Salida	Representa la lectura de datos en la entrada y la impresión de datos en la salida
	Proceso	Representa cualquier tipo de operación
	Decisión	Nos permite analizar una situación, con base en los valores verdadero y falso



1. Problema:
2. Solución:

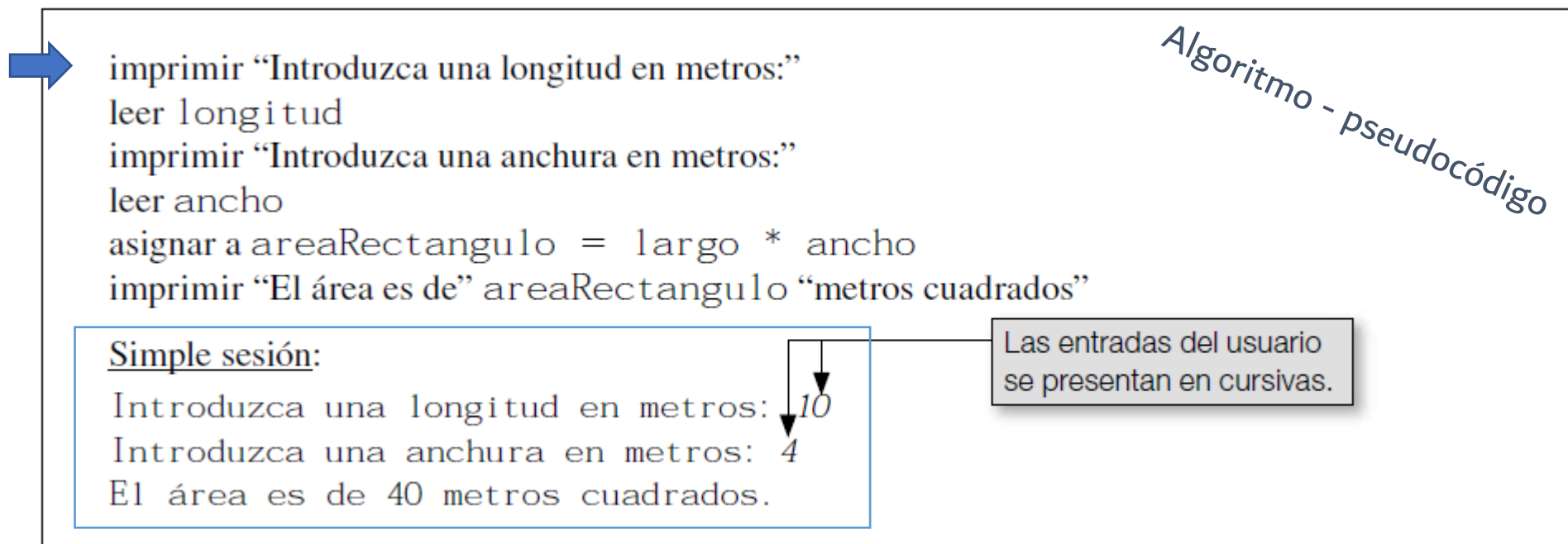
¿Determinar el **área** de un **rectángulo**?

$$\begin{aligned}\text{areaRectangulo} &= \text{largo} * \text{ancho} \text{ (metros)} \\ &= 10 * 4 = \mathbf{40 \text{ metros cuadrados}}\end{aligned}$$

1. Problema

2. Solución

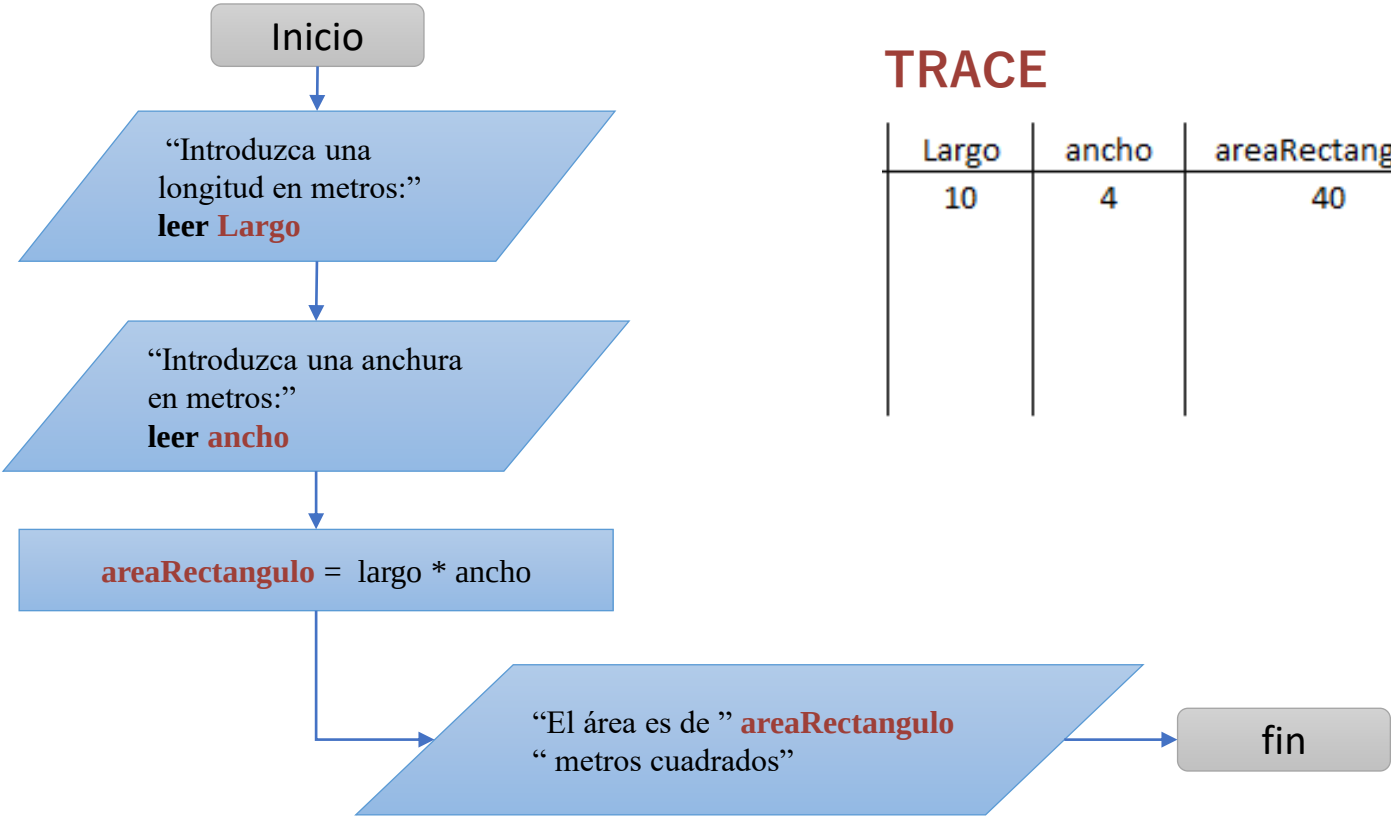
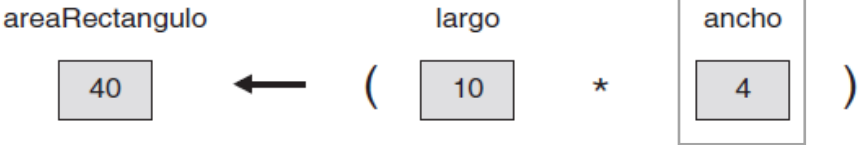
- Algoritmo (pseudocódigo)
- Diagrama de flujo
- Código (debugging)
- **Traza** (Trace) – prueba de escritorio





Algoritmia: pseudocódigo, Diagrama de flujo, Trazabilidad

```
imprimir "Introduzca una longitud en metros:"  
leer longitud  
imprimir "Introduzca una anchura en metros:"  
leer ancho  
asignar a areaRectangulo = largo * ancho  
imprimir "El área es de" areaRectangulo "metros cuadrados"
```



TRACE

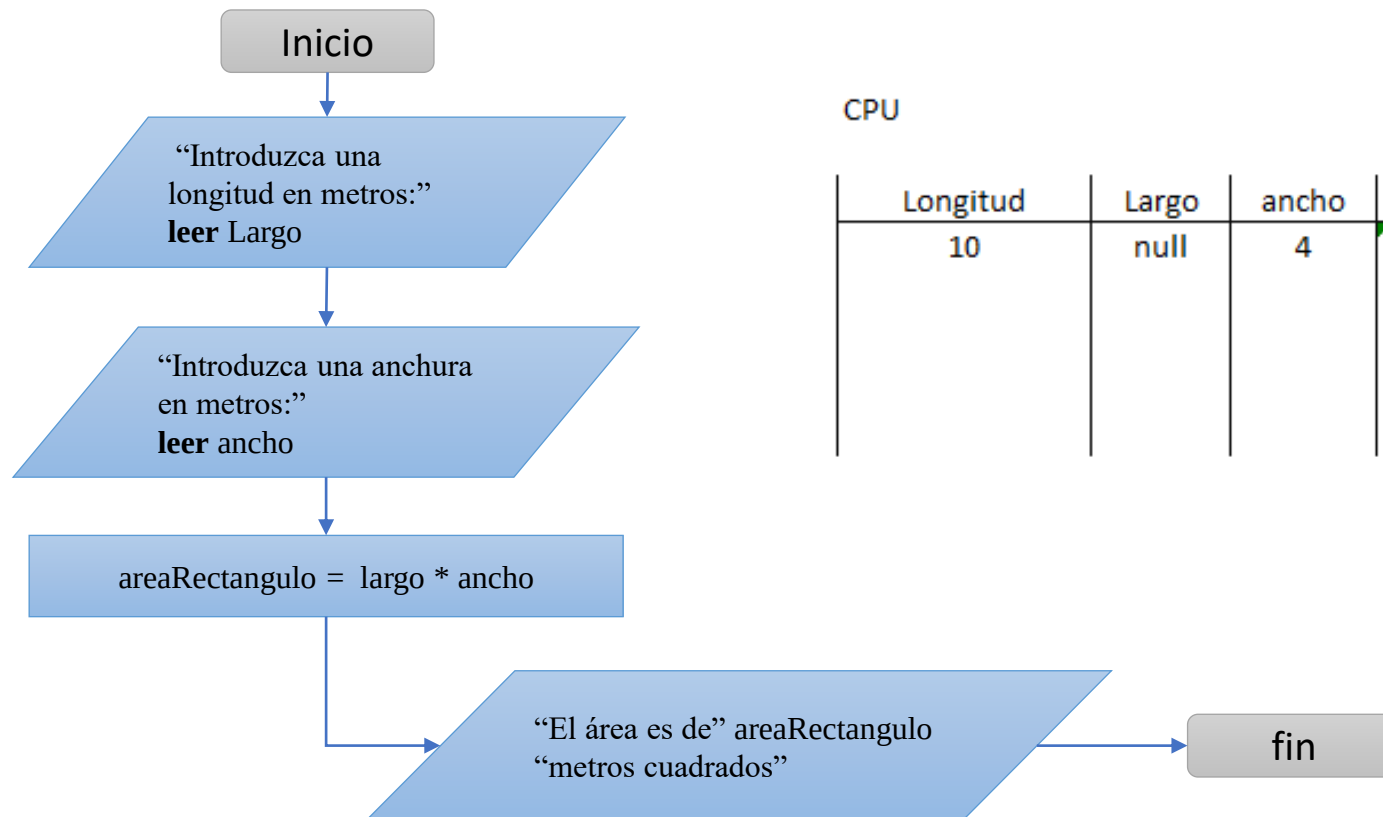
Largo	ancho	areaRectangulo	Salida -> Terminal
10	4	40	Introduzca una longitud en metros 10 Introduzca una anchura en metros 4 El área es de 40 metros cuadrados

ISSUE < > ERROR < > BUG

imprimir “Introduzca una longitud en metros:”
leer longitud
➔ imprimir “Introduzca una anchura en metros:”
leer ancho
asignar a `areaRectangulo` = `largo` * `ancho`
imprimir “El área es de” `areaRectangulo` “metros cuadrados”

INTERPRETE < > COPILADOR

.exe
.class



CPU

Longitud	Largo	ancho	areaRectangulo	Salida -> Terminal
10	null	4	#VALUE!	Introduzca una longitud en metros 10 Introduzca una anchura en metros 4 El área es de <null> metros cuadrados



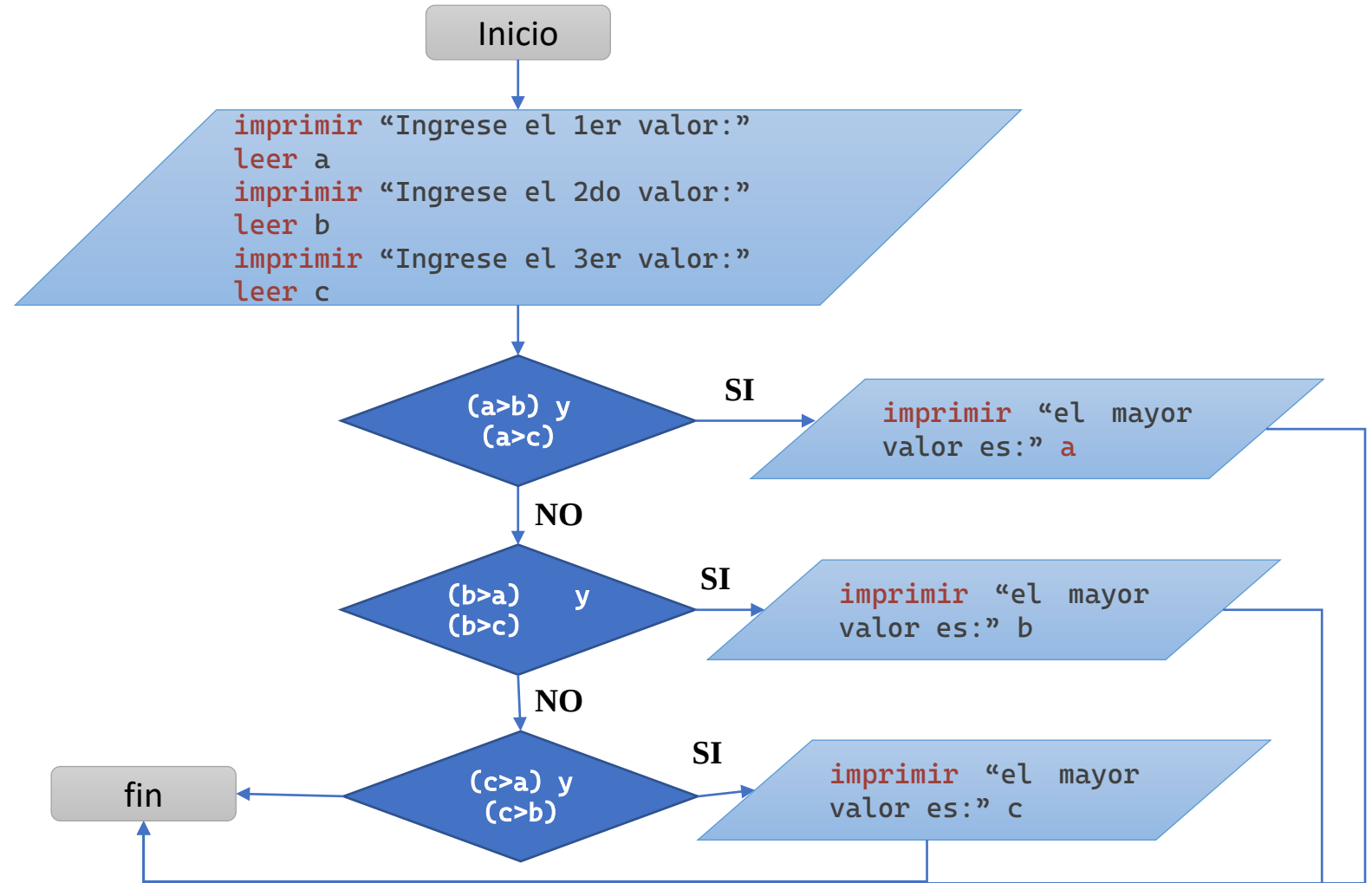
Ejercicio

1. Problema : ¿Cuál es el mayor de 3 números?
2. Solución : 10, **30**, 5

VARIABLES : a =10, b = 30, c=5

- Algoritmo: (pseudocódigo)

```
➔ imprimir "Ingrese el 1er valor:"  
  leer a  
  imprimir "Ingrese el 2do valor:"  
  leer b  
  imprimir "Ingrese el 3er valor:"  
  leer c  
  
Si (a>b) y (a>c)  
  imprimir "el mayor valor es:" a  
  Salir  
Si (b>a) y (b>c)  
  imprimir "el mayor valor es:" b  
  salir  
Si (c>a) y (c>b)  
  imprimir "el mayor valor es:" c
```





Ejercicios

- 1. Problema : determinar le mayor de 3 números
- 2. Solución : 10, **30**, 5

VARIABLES : a =10, b = 30, c=5

- Algoritmo: (pseudocódigo)

```
➡ imprimir "Ingrese el 1er valor:"  
leer a  
imprimir "Ingrese el 2do valor:"  
leer b  
imprimir "Ingrese el 3er valor:"  
leer c  
  
Si (a>b) y (a>c)  
    imprimir "el mayor valor es:" a  
    salir  
Si (b>a) y (b>c)  
    imprimir "el mayor valor es:" b  
    salir  
Si (c>a) y (c>b)  
    imprimir "el mayor valor es:" c
```

a	b	c	Salida -> Terminal
10	30	5	Ingrese el 1er valor
			10
			Ingrese el 2do valor
			30
			Ingrese el 3er valor
			5
			el mayor valor es: 30

a	b	c	Salida -> Terminal
30	10	5	Ingrese el 1er valor
			30
			Ingrese el 2do valor
			10
			Ingrese el 3er valor
			5
			el mayor valor es: 30





- Diagrama de flujo y su código para dibujar las iniciales de su primer nombre y apellido en consola

- Serie: + - ++ --- ++++++ ----- ...

- Resolver: $\sum_{i=1}^n i$ $1+2+3+\dots+n = \sum_{i=1}^n i$

- Resolver: $\sum_{i=1}^n i^2 = 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$



Si n es un entero positivo entonces

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{Fórmula 2.4.1}$$

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} \quad \text{Fórmula 2.4.2}$$

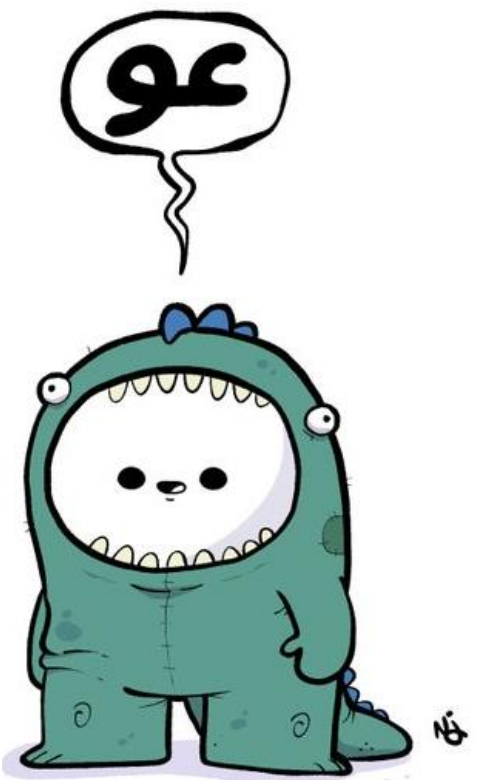
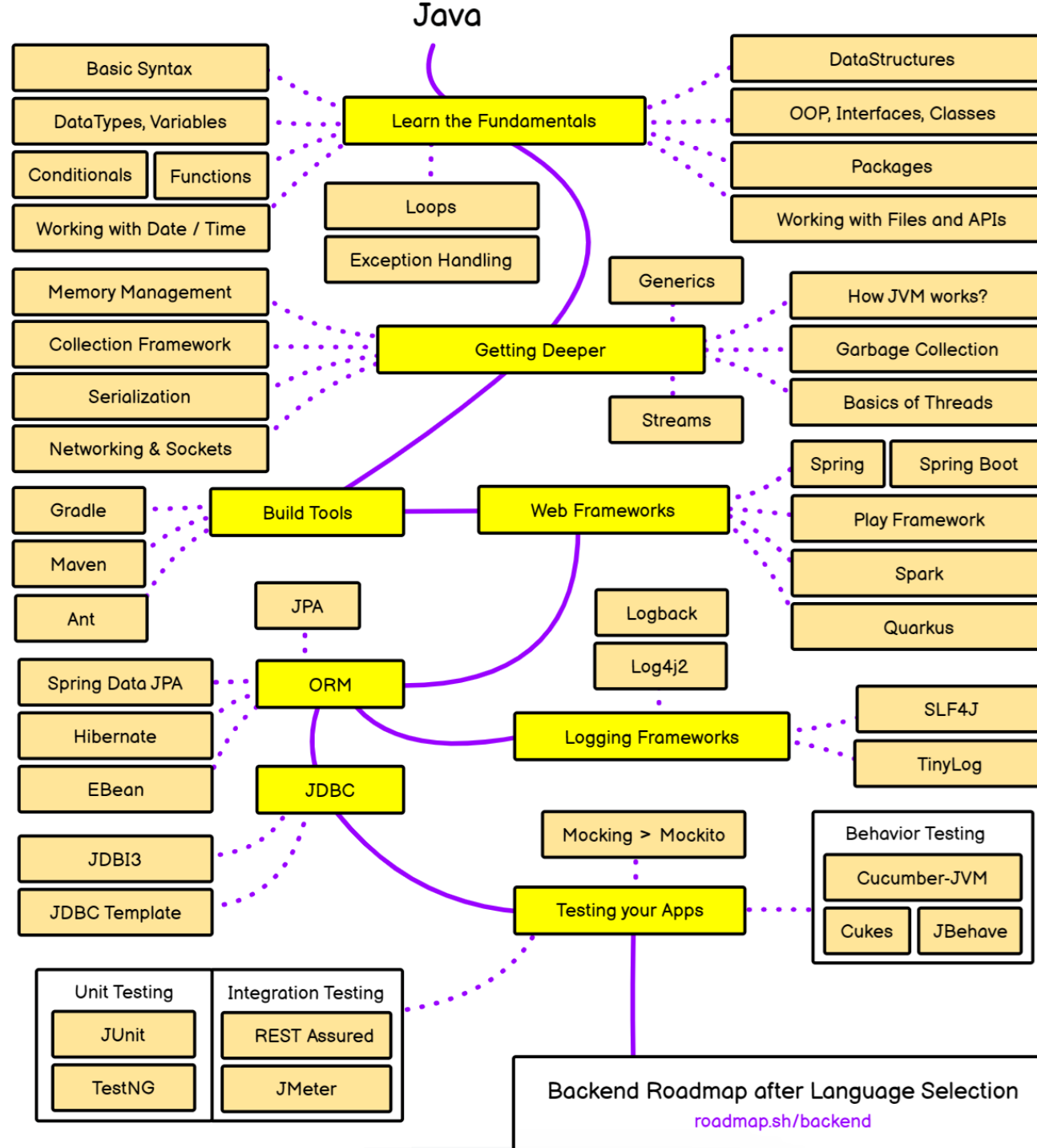
$$\sum_{i=1}^n i^3 = \left[\frac{n(n+1)}{2} \right]^2 \quad \text{Fórmula 2.4.3}$$

$$\sum_{i=1}^n i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30} \quad \text{Fórmula 2.4.4}$$

http://mat.uson.mx/~jldiaz/NotasCD-1/Series/New_Topic.htm



Java Developer Roadmap





Basic Syntax

Understanding the basics is the key to a solid foundation. In this section, learn the basic terminologies, naming conventions, reserved words, conditions, functions, data structures, OOP, packages, etc.

- To print output use —> `System.out.println();`
- To take input from user —> Scanner or BufferedReader class can be used

Visit the following resources to learn more:

- [VIDEOJava - Basic Syntax](#)
- [VIDEOJava Tutorial for Beginners](#)

Data Types and Variables

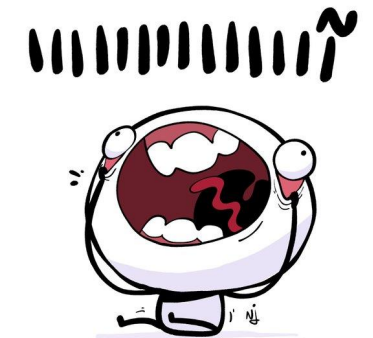
Variable in Java is a data container that stores the data values during Java program execution. Every variable is assigned a data type, which designates the type and quantity of values it can hold. Variable is a memory location name of the data. The Java variables have mainly three types: Local, Instance and Static.

Data Types are divided into two group -

- Primitive - byte,short,int,long,float,double,boolean and char
- Non-Primitive - String, Arrays, Classes, Enums and Records

Visit the following resources to learn more:

- [ARTICLEWhat are Data Types & Variables?](#)
- [ARTICLEJava Variables](#)
- [ARTICLELearn more about Data types and Variables](#)
- [ARTICLEJava enums](#)
- [ARTICLEJava Records](#)



Basic Syntax

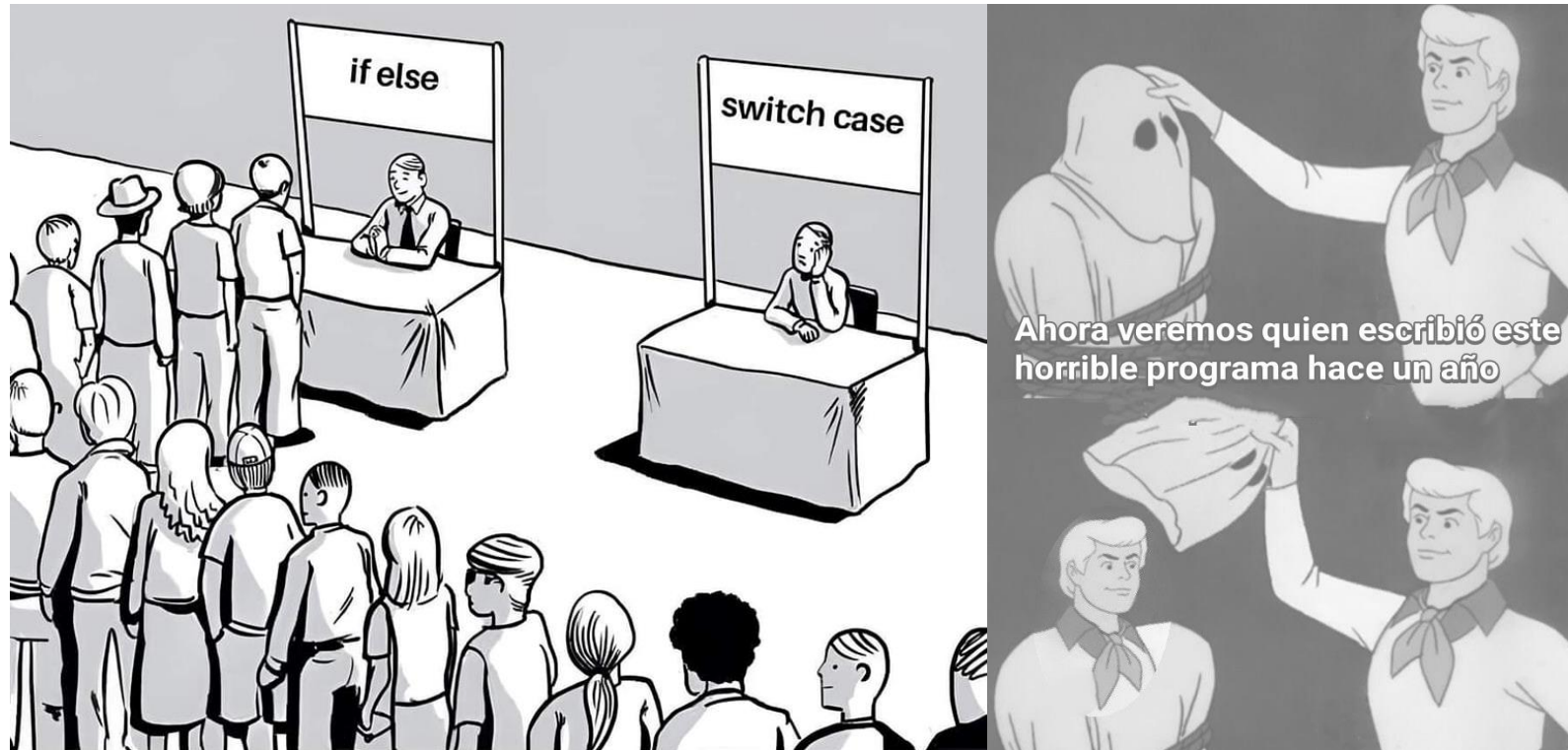
DataTypes, Variables



Conditionals

Java has the following conditional statements:

- Use **if** to specify a block of code to be executed, if a specified condition is true
- Use **else** to specify a block of code to be executed if the same condition is false
- Use **else if** to specify a new condition to test; if the first condition is false
- Use **switch** to specify many alternative blocks of code to be executed
- Use **?:** operator to specify one line condition





method/function

A method/function is a way to perform some task. Similarly, in programming like Java, a function method is a block of code written to perform a specific task repeatedly.

Steps

1. Define function - datatype function_name(parameters){body}

2. Call function - function_name(values)

3. Lambda functions - $x \rightarrow x + 1$

4. Pass a function as a variable -

```
final Consumer<Integer> simpleReference1 = App::someMethod1 simpleReference1.accept(1);
```

Visit the following resources to learn more:

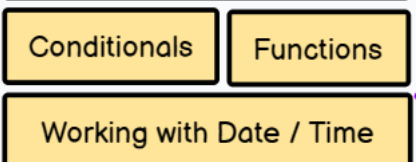
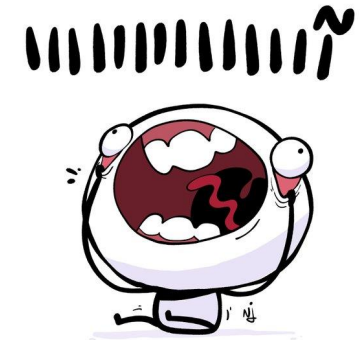
- [ARTICLE Methods/Functions in Java.](#)
- [ARTICLE Learn Functions/Methods in Java](#)
- [VIDEO Functions / Methods in Java](#)
- [VIDEO Lambda functions](#)
- [VIDEO Passing functions as a variable](#)

Date and Time in Java

Visit the following resources to learn more:

- [ARTICLE Date and Time API in Java](#)
- [ARTICLE Introduction to Date and Time in Java](#)
- [ARTICLE Java SE 8 Date and Time](#)

It provides reusability of code. We write the function once and use it many times. It works on the 'DRY' principle i.e., "Do not repeat yourself".





<public/protect/private>

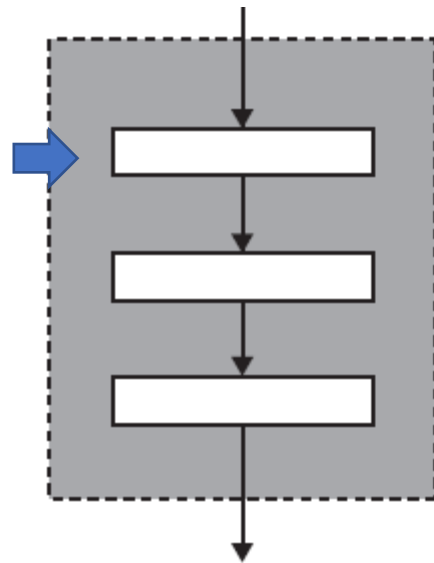
- Métodos
`<public/protect/private> void NombreMetodo() {...}`
- Métodos con parámetros
`<public/protect/private> void NombreMetodo(para1, param2, ..) {...}`

`para1 = <TipoDato> NombreParametro`
- Métodos con retorno
`<public/protect/private> < TipoDato > NombreMetodo()
{... return < TipoDato >; }`
- Métodos con retorno con parámetros
`<public/protect/private> <TipoDato> NombreMetodo(para1, param1, ..)
{... return <TipoDato>; }`

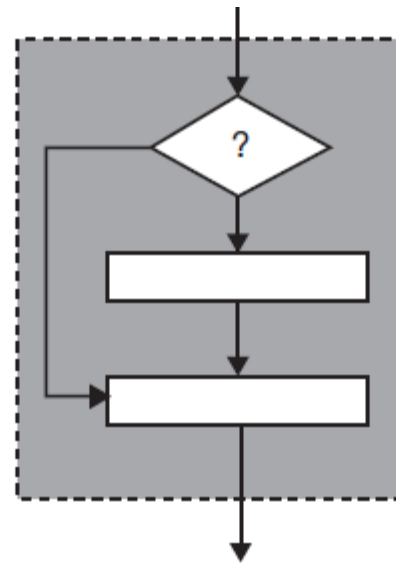
La proposición **for – while – do_While**

Constantes simbólicas

Las estructuras de control

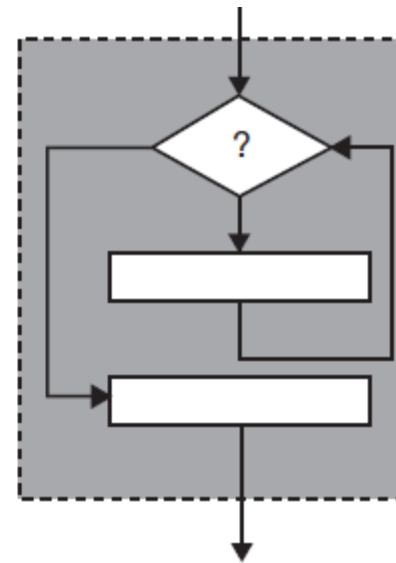


Secuencial



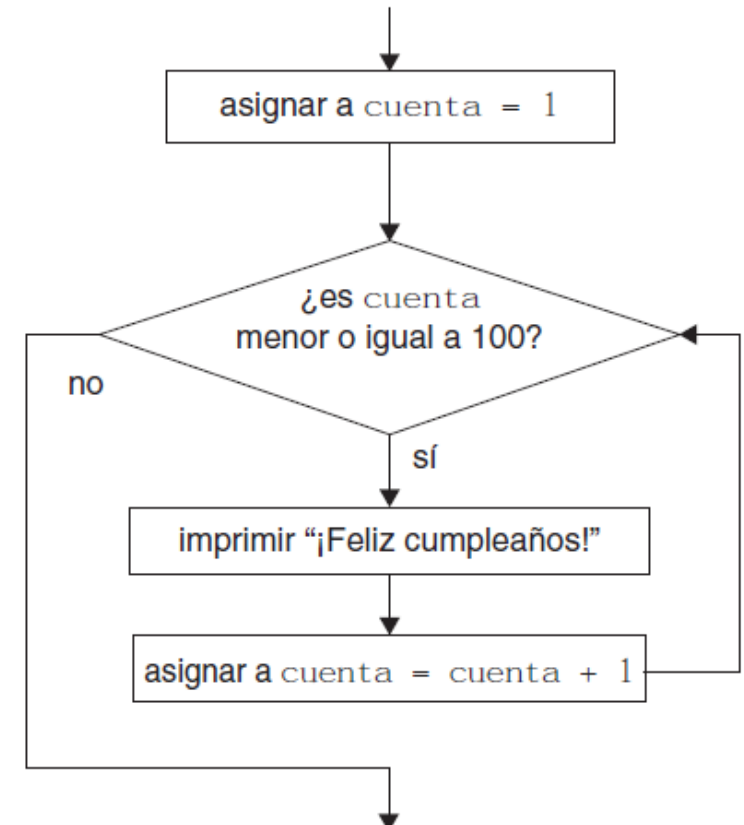
Condicional

“if”
“if, else”
“if, else if”



De control

while <condición>
<sentencia(s)>





Tipos de datos - P.O.O

Tipo de dato	Representación	Tamaño (Bytes)	Rango de Valores	Valor por defecto	Clase Asociada
byte	Numérico Entero con signo	1	-128 a 127	0	Byte
short	Numérico Entero con signo	2	-32768 a 32767	0	Short
int	Numérico Entero con signo	4	-2147483648 a 2147483647	0	Integer
long	Numérico Entero con signo	8	-9223372036854775808 a 9223372036854775807	0	Long
float	Numérico en Coma flotante de precisión simple Norma IEEE 754	4	$\pm 3.4 \times 10^{-38}$ a $\pm 3.4 \times 10^{38}$	0.0	Float
double	Numérico en Coma flotante de precisión doble Norma IEEE 754	8	$\pm 1.8 \times 10^{-308}$ a $\pm 1.8 \times 10^{308}$	0.0	Double
char	Carácter Unicode	2	\u0000 a \uFFFF	\u0000	Character
boolean	Dato lógico	-	true ó false	false	Boolean
void	-	-	-	-	Void

```
int myNum = 100000;  
System.out.println(myNum);
```

```
long myNum = 150000000000L;  
System.out.println(myNum);
```

```
float myNum = 5.75f;  
System.out.println(myNum);
```

```
double myNum = 19.99d;  
System.out.println(myNum);
```

```
boolean isJavaFun = true;  
boolean isFishTasty = false;  
System.out.println(isJavaFun);    // Outputs true  
System.out.println(isFishTasty);  // Outputs false
```

```
char myGrade = 'B';  
System.out.println(myGrade);
```

```
char myVar1 = 65, myVar2 = 66, myVar3 = 67;  
System.out.println(myVar1);  
System.out.println(myVar2);  
System.out.println(myVar3);
```



```
package test;
```

```
/**
 * Clase con ejemplos de testing de java
 * @author:      pat_mic
 * @version:     1.0
 * @see <a href = "http://www.abc.com" /> pat_mic_github </a>
 */
public class Testing {

    /**
     * valida la compilación del java
     */
    public static void TestJava(){
        System.out.printf("java are ready..!");
    }

    /**
     * valida la compilación del java con saludo
     * @param nombre: escribe tu nombre
     * @param fecha : es la fecha actual
     * @return saludo
     */
    public static String TestJavaSaluda(String nombre, String fecha){
        return "java te saluda "+ nombre + " " + fecha ;
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}

System.out.println(2 * 5);

// This is a comment
System.out.println("Hello World");

System.out.println("Hello World"); // This is a comment
```

```
int a=10;
int b=10;

if(a == b++)
    System.out.printf(" \n a y b son iguales");
else
    System.out.printf(" \n a y b no son iguales");
```

//// CAST (explicito, implícito)

```
x++;
x += 1;

int    i = 10;
float  f = 20.2f;

//cast : explicito
i = (int)f;
f = (float)i;

//cast : implicito
f = i/f;

// ejemplos:
i = (int) (i/f);

f = (float)i/f;

f = (int)i/f;
```

CONVERSIÓN

```
int a = Integer.parseInt("10");
int a = Integer.parseInt("234j@");

Double.parseDouble();
```



Variables y secuencias de escape

```
char ch;  
ch = 'A';
```

Secuencias de escape

```
\t mover el cursor al siguiente tabulador  
\n salto de línea (enter)  
\r avanza a la primera columna en el renglón actual  
\\" imprime una literal que utiliza comilla doble  
' imprime una literal con comilla sencilla  
\\ imprime una diagonal invertida
```

Variables primitivas

```
int, long (tipos enteros)  
float, double (tipos de punto flotante)  
char (tipo carácter)
```

Variables de referencia

```
String name = "Thanh Nguyen";  
Calendar fechaNac;
```

In Java, there are different **types** of variables, for example:

- **String** - stores text, such as "Hello". String values are surrounded by double quotes
- **int** - stores integers (whole numbers), without decimals, such as 123 or -123
- **float** - stores floating point numbers, with decimals, such as 19.99 or -19.99
- **char** - stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes
- **boolean** - stores values with two states: true or false

```
String name = "John";  
System.out.println(name);
```

```
int myNum = 15;  
System.out.println(myNum);
```

Example

```
int myNum = 5;  
float myFloatNum = 5.99f;  
char myLetter = 'D';  
boolean myBool = true;  
String myText = "Hello";
```



The general rules for naming **variables** are:

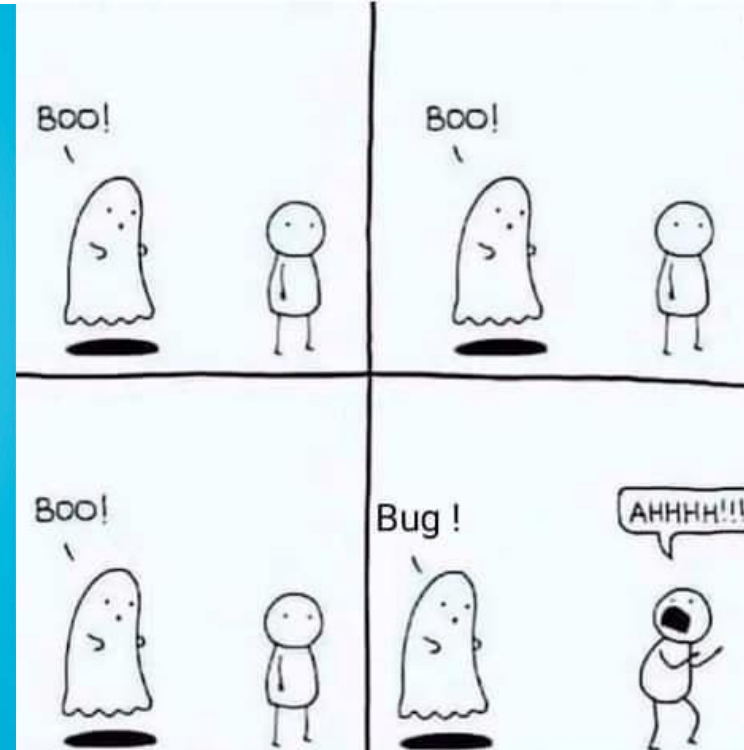
- Names can contain letters, digits, underscores, and dollar signs
- Names must begin with a letter
- Names should start with a lowercase letter and it cannot contain whitespace
- Names can also begin with \$ and _ (but we will not use it in this tutorial)
- Names are case sensitive ("myVar" and "myvar" are different variables)
- Reserved words (like Java keywords, such as int or boolean) cannot be used as names

Example

```
int myNum = 5;           // Integer (whole number)
float myFloatNum = 5.99f; // Floating point number
char myLetter = 'D';     // Character
boolean myBool = true;   // Boolean
String myText = "Hello"; // String
```

Data types are divided into two groups:

- Primitive data types - includes **byte**, **short**, **int**, **long**, **float**, **double**, **boolean** and **char**
- Non-primitive data types - such as String, Arrays and Classes





String : str

```
charAt(index)    // obtener un carácter
length           // obtener la longitud del string
equals           // comparación
equalsIgnoreCase // ignorar si es mayúsculas o minúscula
```

Scanner : std

```
Scanner stdIn = new Scanner(System.in);
```

```
<variable> = stdIn.nextLine();
```

nextInt() Se salta los espacios dejados en blanco hasta que encuentra un valor de tipo int

nextLong() Se salta los espacios dejados en blanco hasta que encuentra un valor de tipo long

nextFloat() Se salta los espacios dejados en blanco hasta que encuentra un valor de tipo float

nextDouble() Se salta los espacios dejados en blanco hasta que encuentra un valor de tipo double

“salta los espacios dejados en blanco ”

next() Se salta los espacios dejados en blanco hasta que encuentra un token. Devuelve el token como un valor tipo String.

Example

```
// Good
int minutesPerHour = 60;

// OK, but not so easy to understand what m actually is
int m = 60;
```





Datastructures

a **Data Structure** is a way of organizing the data in the **memory** so that it can be used efficiently. Some common data structures are array, linked list, stack, hashtable, queue, tree, heap, and graph.

- Array allocates continuous memory for homogeneous data
- Linked List stores data in nodes with references
- Stack follows Last In First Out principle
- Queue follows First In First Out principle

Visit the following resources to learn more:

- [ARTICLE Data Structures and Algorithms](#)
- [VIDEO Data Structures Illustrated](#)

Package, is a namespace that mainly contains classes and interfaces. For instance, the standard class **ArrayList** is in the package `java.util`. For this class, `java.util.ArrayList` is called its fully qualified name because this syntax has no ambiguity. Classes in different packages can have the same name. For example, you have the two classes `java.util.Date` and `java.sql.Date`, which are different. If no package is declared in a class, its package is the default package.

Files and APIs

Learn how to work with files i.e., reading, writing and deleting, files and folders, etc. Also, learn how to make API calls, parse the incoming response, and so on.

- `FileWriter` - this class is useful to create a file by writing characters into it
- `FileReader` - this class is useful to read data in form of characters from file
- `Files.lines(Paths.get("file.txt"))` - processing the files as a stream. Since Java 8
- `Files.readString` / `Files.writeString` - reads the whole file and puts it into a string - since Java 11

Visit the following resources to learn more:

- [ARTICLE How To Work With Files In Java](#)
- [ARTICLE Java File Class](#)
- [ARTICLE New Java HttpClient](#)
- [ARTICLE 5 ways to make HTTP requests in Java](#)
- [ARTICLE Read a file line by line in Java](#)
- [ARTICLE Various ways to read a file to String in Java](#)



DataStructures
OOP, Interfaces, Classes
Packages
Working with Files and APIs



ARRAYS

ARRAYS – BUFER – VECTOR - STRING : cadena de caracteres

```
// caracteres + cadena de caracteres : string / array / vector
char letra = 'a';           // 1 2 3 q ` # $ % 6 & \n \t \r %%
char caracter = '1';        // --> |1| (8bit = 1 byte) --> |0|0|0|1|1|1|
char palabra[10] = "HOLA";  // |H|O|L|A| | | | | | --> vector
char palabras[] = "Como estas pepe...";
String palabras;    → palabras[]
```

Entrada desde consola:

```
c = charAt();
while (c != EOF) { ... }
```

```
while ((c = charAt()) != EOF) { ... }
```

```
while (c = charAt() != EOF)           → c = (charAt() != EOF)   X //precedencia símbolos / operadores
```

```
char s[],c;      // (--)&&(--)||(--)
for (int i = 0; (i < 5) && (c = charAt())!= EOF && (c != '\n') ; ++i) //precedencia condición
    s[i] = c;
```



Varios

```
boolean fingerArriba = true, fingerBajo = false;
```

```
if(fingerArriba)
    System.out.printf(" todo posi 👍 ");
```

```
if(!fingerBajo)
    System.out.printf(" nooo00 negativo ya eres posiii 👍 ");
```

Java Type Casting

Type casting is when you assign a value of one primitive data type to another type. In Java, there are two types of casting:

- **Widening Casting** (automatically) - converting a smaller type to a larger type size
byte -> short -> char -> int -> long -> float -> double

- **Narrowing Casting** (manually) - converting a larger type to a smaller size type
double -> float -> long -> int -> char -> short -> byte



```
public class Main {
    public static void main(String[] args) {
        int myInt = 9;
        double myDouble = myInt; // Automatic casting: int to double

        System.out.println(myInt);    // Outputs 9
        System.out.println(myDouble); // Outputs 9.0
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        double myDouble = 9.78d;
        int myInt = (int) myDouble; // Manual casting: double to int

        System.out.println(myDouble); // Outputs 9.78
        System.out.println(myInt);    // Outputs 9
    }
}
```



Arithmetic Operators

Operator	Name	Description	Example
+	Addition	Adds together two values	$x + y$
-	Subtraction	Subtracts one value from another	$x - y$
*	Multiplication	Multiplies two values	$x * y$
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	$x \% y$
++	Increment	Increases the value of a variable by 1	$++x$
--	Decrement	Decreases the value of a variable by 1	$--x$

Operator	Name	Example
&&	Logical and	$x < 5 \ \&\& \ x < 10$
	Logical or	$x < 5 \ \ x < 4$
!	Logical not	$!(x < 5 \ \&\& \ x < 10)$

Operator	Example	Same As
=	$x = 5$	$x = 5$
+=	$x += 3$	$x = x + 3$
-=	$x -= 3$	$x = x - 3$
*=	$x *= 3$	$x = x * 3$
/=	$x /= 3$	$x = x / 3$
%=	$x \% = 3$	$x = x \% 3$
&=	$x \& = 3$	$x = x \& 3$
=	$x = 3$	$x = x 3$
^=	$x \wedge = 3$	$x = x \wedge 3$
>>=	$x >> = 3$	$x = x >> 3$
<<=	$x << = 3$	$x = x << 3$

Operator	Name	Example
==	Equal to	$x == y$
!=	Not equal	$x != y$
>	Greater than	$x > y$
<	Less than	$x < y$
>=	Greater than or equal to	$x >= y$
<=	Less than or equal to	$x <= y$

```
int x = 5;
int y = 3;
System.out.println(x == y); // ????
```

you can do it 😊



```
/**
 * Clase con ejemplos de testing de java
 * @author:      pat_mic
 * @version:     1.0
 * @see <a href = "http://www.abc.com" /> pat_mic_github </a>
 */
```

```
if (args.length != 3) {
    System.out.println(x:"No me has dado bien los parámetros");
    return;
}
if (!args[1].equals(anObject:"+") && !args[1].equals(anObject:"-")) {
    System.out.println(x:"No sé lo que me has pedido");
    return;
}

System.out.print(args[0] + " " + args[1] + " " + args[2] + " = ");
int a = Integer.parseInt(args[0]);
int b = Integer.parseInt(args[2]);
if (args[1].equals(anObject:"+")) {
    System.out.println(a + b);
} else if (args[1].equals(anObject:"-")) {
    System.out.println(a - b);
}
```

```
var str = Arrays.toString(args);
System.out.println(str);
```

```
Scanner scan = new Scanner(source: System.in);

int total = 0;
while (scan.hasNext()) {
    total += scan.nextInt();
}
System.out.println(x: total);
```

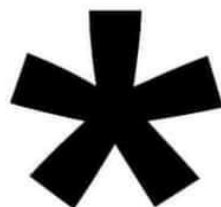
```
String datos = "1234I5678-9012";
Scanner scan = new Scanner(source: datos);

scan.useDelimiter(pattern: "-");

int total = 0;
while (scan.hasNext()) {
    total += scan.nextInt();
}
```



```
import java.util.Array  
import java.util.ArrayLis  
import java.util.Date  
import java.util.logging.  
import java.util.logging.  
import java.util.math  
import java.util.random  
import java.util.scanner
```







Hoy es tu oportunidad de demostrar de lo que estas hecho...



WANTED

Dark Man



"Su poderoso físico y conocimiento en programación le convierte en un auténtico tanque humano. Además, su dominio del Overwatch desesperaba a todo el que osaba retarle."

REWARD: 1000 pt

WANTED

Super Woman



"Su atronadora voz resonaba en todo el valle y era capaz de dominar poblados enteros con su conocimiento de programación"

REWARD: 5.000 pt

WANTED

Dark Woman



"Las extrañas palabras y guturales sonidos que salían de su boca confundían a los que no saben programar con lo cual te humilla"

REWARD: 2500 pt



Misión → Conocer nuestro estado inicial en la programación



*Para el ganador
2 extras*





Misión → Conocer nuestro estado inicial en la programación



pat_mic

*"maestro del arte de
la programación a
bajo nivel y del
oscuro mundo del
la programación
orientada a objetos
POO"*

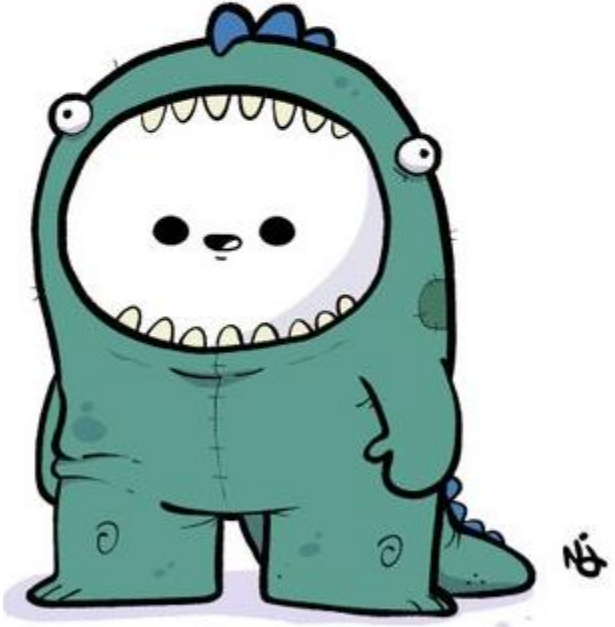
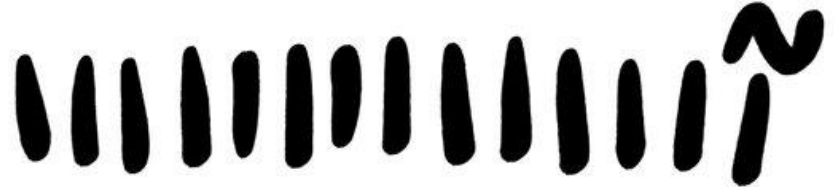
*pat_mic, busca el saber de sus nuevos
guerreros para la travesía en el saber y arte
de la programación.*

*Propone realizar un duelo de conocimiento de
habilidades ocultas para programar la famosa
magic loading-bar en la terminal:*

MAGIC LOADING...



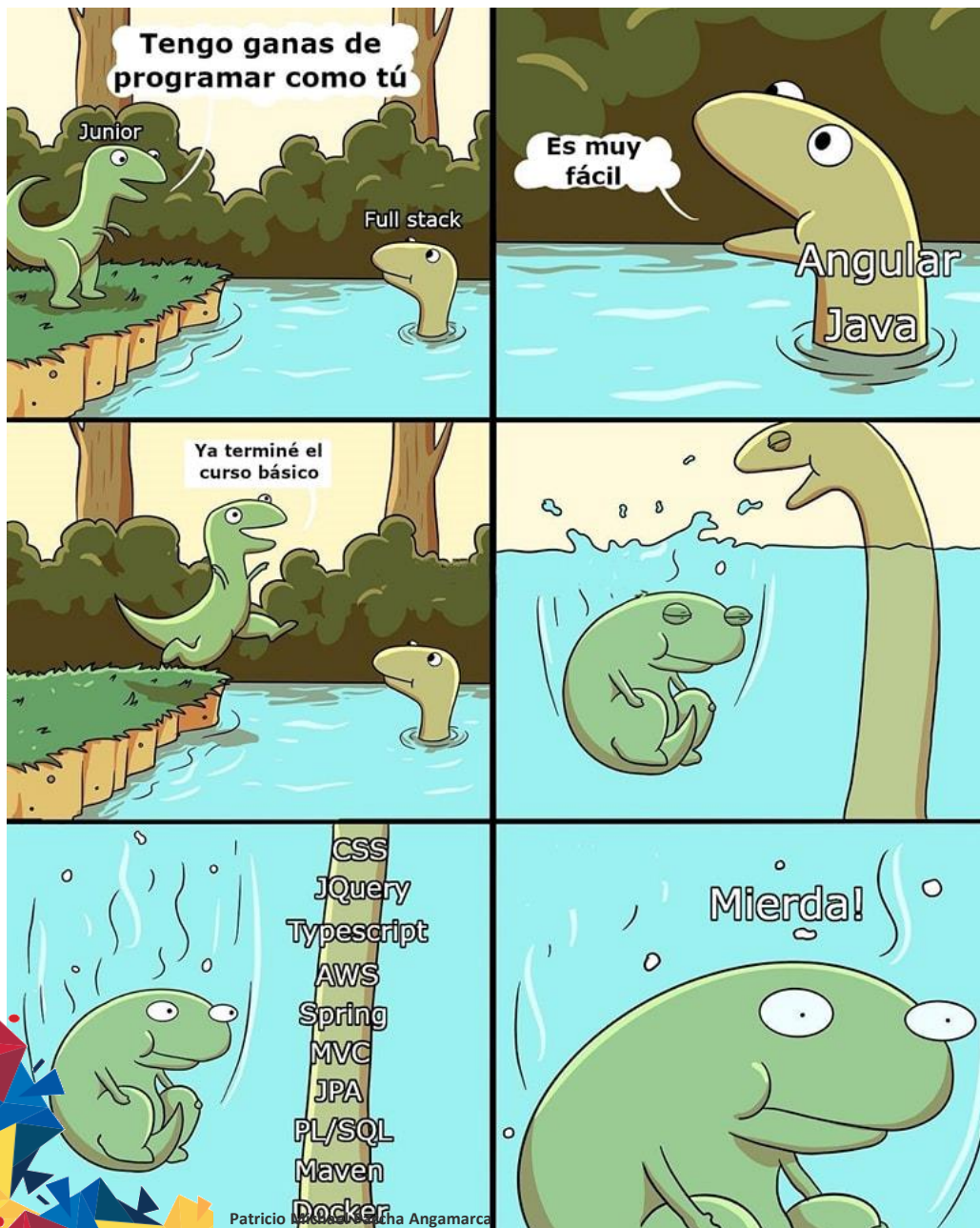
75 %





MAGIC LOADING...





```
/**  
 * Clase con ejemplos de testing de java  
 * @author: pat_mic  
 * @version: 1.0  
 * @see <a href = "http://www.abc.com" /> pat_mic_github </a>  
 */
```

Workshop

PATMIC - RETOS

- Señal
- Torres de Hanoi
- Lobito & Caperucita
- Poli Retos



2 Introduction to Java Applications; Input/Output and Operators

- 2.1 Introduction
- 2.2 Your First Program in Java: Printing a Line of Text
- 2.3 Modifying Your First Java Program
- 2.4 Displaying Text with `printf`
- 2.5 Another Application: Adding Integers
 - 2.5.1 `import` Declarations
 - 2.5.2 Declaring Class Addition
 - 2.5.3 Declaring and Creating a Scanner to Obtain User Input from the Keyboard
 - 2.5.4 Declaring Variables to Store Integers
 - 2.5.5 Prompting the User for Input
 - 2.5.6 Obtaining an `int` as Input from the User
 - 2.5.7 Prompting for and Inputting a Second `int`
 - 2.5.8 Using Variables in a Calculation
 - 2.5.9 Displaying the Result of the Calculation
 - 2.5.10 Java API Documentation
- 2.6 Memory Concepts
- 2.7 Arithmetic
- 2.8 Decision Making: Equality and Relational Operators
- 2.9 Wrap-Up



PROGRAMACIÓN ORIENTADA A OBJETOS

FACULTA DE INGENIERÍA EN INFORMÁTICA
ESCUELA POLITÉCNICA NACIONAL

Patricio Michael Paccha Angamarca
Magister en ingeniería de software